The Journal of Systems and Software 230 (2025) 112540

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

On the adoption of software bill of materials in open-source software projects☆

Sabato Nocera [a],*, Simone Romano [a], Massimiliano Di Penta [b], Rita Francese [a], Giuseppe Scanniello [a]

[a] University of Salerno, Department of Computer Science, Fisciano, Italy
[b] University of Sannio, Department of Engineering, Benevento, Italy

ARTICLE INFO

Keywords:
Software bill of materials
Software supply chain
Empirical study

ABSTRACT

A *Software Bill of Materials* (*SBOM*) formally lists the open-source and proprietary components that constitute a software product, including their licenses, versions, vendors, vulnerabilities, and supply chain relationships. SBOMs enable software producers and consumers to gain visibility into the software supply chain and monitor the risks associated with software security, licensing, and more. This paper presents the results of an exploratory mining study investigating the adoption of SBOMs by open-source software projects. To that end, we mined *GitHub* and identified 186 public software repositories using SBOM generation tools owned by *SPDX* and *CycloneDX*. Although the adoption of SBOMs is low, it is increasing. Moreover, SBOMs are under version control or available in public release versions of less than half the software projects analyzed. Finally, only a limited fraction of SBOMs contain minimum/recommended information, and some SBOMs are also uncompliant with existing SBOM standards. Our study reveals that software producers are paying more attention to SBOMs, but even so, these may be incomplete. We urge software producers to adopt SBOMs and meet the new software supply chain standards. As for researchers, we foster further investigations on adopting SBOMs and their correct use.

1. Introduction

A *Software Bill of Materials* (*SBOM*) is a machine-readable record containing the details and supply chain relationships of the open-source and proprietary components used in building a software artifact (United States Government, 2021). For instance, an SBOM can detail all information on the libraries employed by a software product, including their licenses, versions, vendors, and vulnerabilities. SBOMs originate from manufacturing, where *Bill of Materials* (*BOMs*) let producers monitor the components used to make a product: whenever a defect is identified in a component, a BOM makes it simple to identify the affected component and solve the issue, possibly alerting the users. SBOMs apply the same approach to software products, allowing developers to identify, trace, and address not only reliability- and security-associated risks but also legal ones originating from integrating components with incompatible licenses.

More concretely, SBOMs introduce potential benefits for stakeholders involved at different levels in software development and usage. For example, software producers can ensure that used (third-party) components are up to date, especially with respect to vulnerability patches. Software buyers can use SBOMs to perform vulnerability or license analysis, both of which can be used to evaluate risks in the processes of selection/acquisition of software products. System administrators can use SBOMs to quickly and easily determine whether they are at potential risk of newly discovered vulnerabilities.

The need to develop a mechanism such as SBOM is probably due to the peculiar nature of modern software products, which no longer consist of only proprietary code. Nowadays, software products are usually developed by leveraging existing open-source and proprietary components, resulting in complex supply chains that are only as secure as their least secure component. Indeed, according to the *Census II* investigation by the *Harvard Laboratory for Innovation Science* and the *Open Source Security Foundation*, 70%–90% of modern software products depend on open-source software (The Linux Foundation, 2022a). Moreover, the *OWASP Top 10*—a document representing a broad consensus about the most critical security risks, regularly updated by

☆ Editor: Tao Zhang.

* Corresponding author.
E-mail addresses: snocera@unisa.it (S. Nocera), siromano@unisa.it (S. Romano), dipenta@unisannio.it (M.D. Penta), francese@unisa.it (R. Francese), gscanniello@unisa.it (G. Scanniello).

https://doi.org/10.1016/j.jss.2025.112540
Received 15 November 2024; Received in revised form 3 June 2025; Accepted 17 June 2025
Available online 1 July 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

OWASP (*Open Source Foundation for Application Security*)—reports the usage of outdated and vulnerable components as one of the most critical security risks (OWASP, 2021). For example, *Heartbleed*, a vulnerability disclosed in 2014 in the popular *OpenSSL* cryptography library, costed businesses at least 500 million USD (eWeek, 2014). Another example is *Log4Shell*, a vulnerability disclosed in 2021 in the popular *Apache Log4j* logging framework, which affected more than 35,000 Java packages (amounting to over 8% of the *Maven* ecosystem), and fixing them in all the affected software products will take years (Google Online Security Blog, 2021). SBOMs would have enabled the immediate notification of these vulnerabilities in the affected software products and, more specifically, in their components, as well as an understanding of whether and to what extent they threatened their security. This is because not all software product vulnerabilities can be exploited by an attacker.

Public administrations have been raising the need for secure and accountable software products. In 2020, *ENISA* (*European Union Agency for Cybersecurity*) delineated a series of guidelines for securing the *Internet of Things* (*IoT*), which include the release of SBOMs for IoT devices (European Union Agency for Cybersecurity, 2020). In 2022, the *European Commission* proposed a cybersecurity regulation outlining that software producers must document the vulnerabilities and components of their software products with SBOMs (European Commission, 2022). In 2021, the *United States Federal Government*, per President Biden's Executive Order 14028, laid down that any organization releasing software products to federal agencies must provide SBOMs for the released software products (United States Government, 2021). Although this executive order targets organizations conducting business with federal agencies, SBOMs are expected to shortly become the *de-facto* standard for any organization that develops or maintains software products in both industrial and open-source contexts. Moreover, the executive order mandated the *United States Department of Commerce* and *National Telecommunications and Information Administration* (*NTIA*)[1] to publish the minimum elements of SBOMs, including the standards to be used for SBOMs creation and consumption—*i.e., SPDX* (*Software Package Data eXchange*), *CycloneDX*, and *SWID* (*SoftWare IDentification*) *tags* (United States Department of Commerce, 2021).

Although the use of SBOMs has been mainly motivated by security-related organizations and enforced by public administrations, the adoption of SBOMs seems to be still limited, as shown by recent studies (The Linux Foundation, 2022b; Zahan et al., 2023; Xia et al., 2023; Stalnaker et al., 2024; Chaora et al., 2023; Kloeg et al., 2024; Nocera et al., 2024a). This could be due to several heterogeneous factors, including limited knowledge, lack of regulations in some countries, and limited tool support, especially for SBOM consumption, e.g., to check, based on SBOM, whether a software is using vulnerable components.

While previous research mainly studied SBOMs based on practitioners' perceptions (*e.g.,* Nocera et al., 2024a), to the best of our knowledge, there is a lack of empirical evidence on the extent to which and how owners of existing software projects generate SBOMs and keep them up-to-date.

In this paper, we present the results of an explorative study to complement the existing literature that studied existing challenges and barriers to adopting SBOMs from a qualitative perspective. Specifically, we mined *GitHub* to understand *(i)* to what extent SBOMs are adopted by open-source software projects; *(ii)* what is their adoption trend; *(iii)* how SBOMs are distributed; *(iv)* with which frequency SBOMs made available in software repositories get updated; *(v)* to what extent they comply with the standards; and *(vi)* what pieces of information SBOMs report, including (United States Department of Commerce and) NTIA's minimum and recommended data fields (NTIA, 2021g).

[1] It is an Executive Branch agency of the United States Department of Commerce that serves as President's principal adviser on telecommunications and information policy issues.

We identified 186 open-source repositories — *repositories* and *projects* are terms we use interchangeably in this paper — on GitHub that generate their SBOMs in *SPDX* or *CycloneDX*, which are the two most widely used SBOM standards (Xia et al., 2023). Each repository generates its SBOM using at least one SBOM generation tool among 19 owned by SPDX or CycloneDX. Even though SBOM adoption in the analyzed open-source repositories is still pretty low, we observed that it is trending upward. This may result from increased pressure and interest from major players like the European Union and the United States Government. At the same time, we found that in 46% of the software projects examined, SBOMs are under version control or available in public release versions (sometimes, the same repository can distribute multiple SBOMs). Also, we found that 12% of the analyzed SBOMs do not comply with the corresponding standard, and only 7% of them report information for all NTIA's minimum data fields. As for NTIA's recommended data fields, no information on vulnerabilities was ever found, and only a few SBOMs report licensing and hash information. Therefore, we acknowledge an urgent necessity for organizations to upgrade their software infrastructure to comply with the latest guidelines mandated for the software supply chain.

This paper extends our previous conference publication (Nocera et al., 2023) as follows:

1. We investigate two further research questions (RQ5 and RQ6) to understand, respectively, the extent to which SBOMs comply with the standards and what pieces of information SBOMs report, including NTIA's minimum and recommended fields (United States Department of Commerce, 2021).
2. We extended the presentation and discussion of the results in light of the newly added research questions, which allowed us to distill nine more findings.

Paper Structure. Section 2 provides some background information about SBOMs and discusses related work. Section 3 describes the study design, while the obtained results are reported in Section 4. A discussion of the results and their implications is reported in Section 5, along with possible threats to their validity. Section 6 concludes the paper and outlines directions for future work

2. Backgrounds and related work

In this section, we first provide some background notions about software supply chains and SBOMs. Then, we discuss related literature about SBOMs, software supply chains, and software dependencies.

2.1. Software supply chains and software bills of materials

A software supply chain consists of anything needed (*e.g.,* components and processes) to develop and deliver a software product. An SBOM describes the inventory of components used in a software product, often composed of open-source and proprietary components (United States Department of Commerce, 2021). That is, an SBOM describes, in a structured and machine-readable format (*e.g.,* XML, JSON), the third-party components that constitute a software product, along with information about their licenses, versions, vendors, vulnerabilities, dependency relationships, *etc.* To some extent, an SBOM can be compared to the "organic logo" (The European Commission, 2025) in food products, which lists all ingredients used to make such food and allows one to follow the whole production chain. An SBOM is useful to both software producers and consumers. For example, software producers often leverage third-party components; therefore, SBOMs should allow them to ensure those components are up-to-date and check for known vulnerabilities. Software supply chain attacks involve the exploitation of vulnerabilities in components on which a software product depends. By leveraging a more mature and informed risk-based process, software producers can more proactively evaluate and remediate security risks

Table 1
Minimum data fields for SBOMs (United States Department of Commerce, 2021).

Data field	Description
Supplier name	The entity responsible for creating, defining, and identifying components.
Component name	The name given to a component by the original supplier.
Component version	The identifier the supplier uses to specify different versions of a component.
Other unique identifiers	Other component identifiers that may reference relevant databases.
Dependency relationship	Description of the dependency relationships among components.
Author of SBOM data	The name of the entity that creates the SBOM.
Timestamp	Record of the date and time of the SBOM generation.

in the software supply chain, thereby improving customers' and users' trust Nocera et al. (2025). For instance, given an SBOM, it is possible to periodically monitor the components described in that SBOM for newly disclosed vulnerabilities affecting such components.

The software supply chain and its security have been recognized as a top priority by both the European Union (ENISA's *Guidelines for Securing the Internet of Things* (European Union Agency for Cybersecurity, 2020) and *Cyber Resilience Act*, not yet enacted, proposed by the European Commission (2022)) and the United States Government (President Biden's *Executive Order 14028 on Improving the Nation's Cybersecurity* United States Government, 2021). Despite having received notable policy attention, the software industry has yet to implement SBOMs fully (Zahan et al., 2023). In this respect, the United States Government was the first authority to show interest in establishing guidelines to facilitate the adoption of SBOM. In particular, President Biden's Executive Order 14028 mandated the United States Department of Commerce and NTIA to publish the minimum elements for SBOMs (United States Department of Commerce, 2021), which comprise the following interrelated areas:

- *Data Fields*, indicating the baseline information (see Table 1) about each component to enable its tracking across the software supply chain and its mapping to other sources of data (*e.g.,* vulnerability or license databases).
- *Automation Support*, related to software tools for producing or consuming SBOMs, including the SBOM standards (*i.e.,* SPDX, CycloneDX, and SWID tags).
- *Practices and Processes*, devising strategies to integrate SBOMs into the software development life cycle—*e.g.,* a new SBOM must be produced and made available to those who need it whenever a component is updated or new information on that component becomes available like disclosed vulnerabilities.

In addition to the minimum elements for SBOMs, the United States Department of Commerce and NTIA (United States Department of Commerce, 2021) published recommended data fields. They are:

- *Component Hash*, a fixed-size string generated by a hash function from a component's binary or source code, uniquely identifying the component and verifying its integrity.
- *Life Cycle Phase*, indicating when (*e.g.,* pre-, post-, or within-building) and from where (*e.g.,* source or binary code) the data reported in the SBOM was collected.
- *Other Component Relationships*, indicating kinds of relationship (*e.g.,* derivation or descendant) different from the dependency relationship.
- *License Information*, needed to manage licensing risks due to employing components with incompatible licenses.

From here onwards, we refer to the data fields listed in Table 1 and those listed above as NTIA's minimum and recommended data fields, respectively.

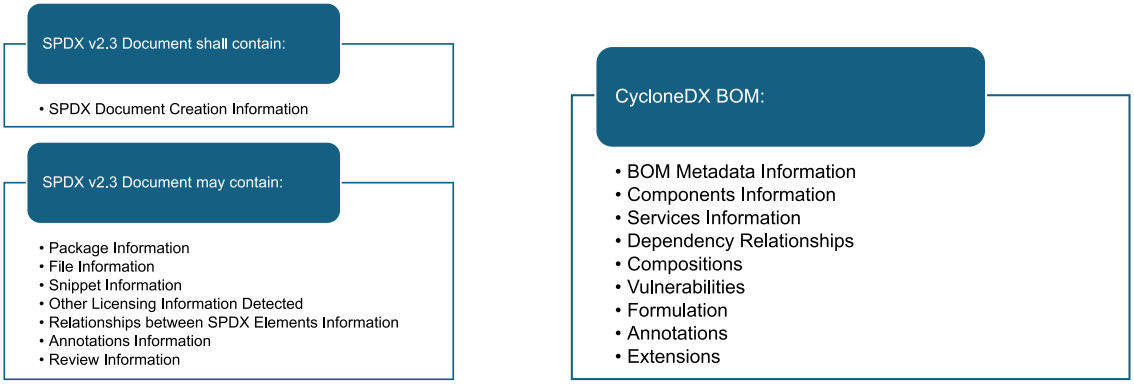
As for vulnerabilities, although there are SBOM standards that allow storing vulnerability information inside SBOMs, the United States Department of Commerce and NTIA suggest storing vulnerability information in separate data structures linked to SBOMs (United States Department of Commerce, 2021). This requirement emerges because vulnerability information is dynamic and evolves over time—a software product previously deemed not secure may become insecure as new vulnerabilities are discovered. As such, vulnerability information in SBOMs cannot be assumed to be complete and up-to-date unless very specific conditions and processes are in place (United States Department of Commerce, 2021).

SBOM generation should produce a comprehensive and accurate list of all the components a software product depends on, which might vary depending on the stage of software development (*e.g.,* considering pre- vs. post-build artifacts) (Nadgowda, 2022). Manual SBOM generation is impractical due to the large number of dependencies a software product can have. Therefore, it is necessary to rely on automated tools, possibly integrated into the development pipeline. For example, some SBOM generation tools can be plugged into package managers. In such a case, the SBOM generation tool is declared among the project's dependencies in the package manager's build/lock files.

Multiple standards have been defined to make SBOMs machine-readable and allow their consumption by other tools. An SBOM standard is a machine-readable schema conceived to provide a uniform structure for describing the composition of a software product so that the SBOM can be consumed by other tools (*e.g.,* vulnerability or license scanners). As pointed out by Xia et al. (2023), the most commonly adopted standards are SPDX (2024b) and CycloneDX (2024b).

SPDX is an open standard with origins in the *Linux Foundation*. Initially conceived to improve software license compliance, SPDX was then extended to support further use cases like those related to security. In 2021, SPDX was approved as an ISO/IEC standard (ISO, 2021). Another open standard for SBOMs is CycloneDX, which was developed by OWASP. Similar to SPDX, it supports different use cases, from software license compliance to security-related ones. The open nature of both SPDX and CycloneDX and the availability of software tools to generate SBOMs complying with these standards should ease their adoption. Our investigation aims to understand if this assumption holds for open-source projects hosted on GitHub.

Both SPDX and CycloneDX incorporate existing specifications to document component identifiers, including *Package URL* (*purl*), *Common Platform Enumeration* (*CPE*), and SWID tags. Indeed, SBOM is the glue that maps external data to software products/components, supporting simple and complex use cases (*e.g.,* mapping to vulnerability databases to retrieve information on disclosed vulnerabilities and their mitigations, or ongoing monitoring and risk analysis of third-party components) (United States Department of Commerce, 2021). The same component can be identified through different kinds of identifiers. Also, a database containing component information might support a certain kind of identifier rather than another. For instance, the *National Vulnerability Database* (*NVD*) supports only CPE. Since SWID tags are considered more of a software identifier than an SBOM standard, we decided to focus our study on SPDX and CycloneDX (Xia et al., 2023). It is worth noting that hashes can also serve as software identifiers. However, they are reported in SBOM in specific data fields dedicated to hash values (unlike other component identifiers); also, there are situations when a hash cannot be generated (*e.g.,* partial or complete absence of the binary/source code from which to generate the hash) or require further information on how it will be generated (*e.g.,* while generating a hash from a source file is straightforward, for an executable file there might be some differences based on the packages employed) (United States Department of Commerce, 2021).



(a) SPDX object model [23]. (b) CycloneDX object model [24].

Fig. 1. SPDX and CycloneDX object models.

SPDX and CycloneDX allow describing virtually the same component information, albeit in different ways and using different terminology. Fig. 1 shows the SPDX and CycloneDX object models, depicting the sections an SBOM must/can have. *SPDX Document Creation Information* and *BOM Metadata Information* are, respectively, the SPDX and CycloneDX sections describing the software product for which the SBOM was produced and how the SBOM was created, by whom, and when. In CycloneDX, the *Components Information* section documents first- and third-party components (including software and hardware components, machine learning models, source code, and configurations) while the *Services Information* documents the usage of external APIs. The same information (*i.e.,* internal components and external services) can be virtually described in SPDX through the *Package Information*, *File Information*, and *Snippets Information* sections. In both SPDX and CycloneDX, licensing information can be described along with the declared components. However, SPDX also provides a section named *Other Licensing Information Detected*. *Relationships between SPDX Elements Information* and *Dependency Relationships* are, respectively, the SPDX and CycloneDX sections outlining direct and transitive dependencies involving first- and third-party components, according to a dependency graph representation. CycloneDX provides a *Vulnerabilities* section to document any information on disclosed vulnerabilities within first- and third-party components; similarly, it is possible to document this information in SPDX along with the component information in the *Package Information*, *File Information*, and *Snippets Information* sections. Compared to other formats, CycloneDX potentially allows for a more in-depth description and characterization of vulnerabilities, *e.g.,* to what extent an attacker could exploit a vulnerability in a component. The reason for this better support is because CycloneDX was explicitly conceived to better support vulnerability management in complex software supply chains through the *Vulnerability Exploitability eXchange* (*VEX*) specification (NTIA, 2021h). A comprehensive description of SPDX and CycloneDX specifications is available on the web (SPDX, 2023b; CycloneDX, 2023f).

The SPDX and CycloneDX standards define the structure of SBOMs; as for how to report component information, it is necessary to rely on other documents, such as Department of Commerce and NTIA's guidelines on minimum elements for SBOMs (United States Department of Commerce, 2021) and the mapping of these elements to the SPDX and CycloneDX standards (NTIA, 2021a). Indeed, these documents outline how to put SBOMs into practice. For instance, SPDX and CycloneDX standards define sections to report dependencies among components but not the minimum *depth* in the dependency graph; in this respect, according to NTIA (United States Department of Commerce, 2021), an SBOM should contain at least all *direct* dependencies among components, as well as all *transitive* dependencies or sufficient details to retrieve transitive dependencies recursively.[2]

The typical file format for SPDX and CycloneDX SBOMs is JSON or XML. However, SPDX SBOMs can also be produced as a plain text file. Fig. 2 provides examples of two SBOMs in JSON format, one is related to the SPDX standard and the other to CycloneDX. Both SBOMs report component information about the same system, *i.e.,* a Java Maven project that imports only one dependency into the `pom.xml` file, namely `Apache Log4j`. The SBOM files were generated using the SPDX and CycloneDX plugins for Maven.

2.2. Empirical work on software bills of material

Linux Foundation's SBOM readiness survey in 2022 (The Linux Foundation, 2022b) on 412 worldwide organizations showed gaps in familiarity with, production planning for, and consumption of SBOMs. For example, 40% of the organizations were unclear about whether the industry is committed to mandating SBOMs, raising concerns about whether the software industry is abiding by President Biden's Executive Order 14028 (United States Government, 2021).

Xia et al. (2023) conducted 17 interviews and a survey with 65 respondents to investigate *(i)* the adoption of SBOM in practice, *(ii)* the SBOM tool support, and *(iii)* the main concerns related to the use of SBOMs. Based on the collected results, the authors derived 26 statements describing the SBOM adoption and its challenges. Those are related, for example, to the still limited awareness, the need for better incentives in using SBOMs, a limited consensus on what should be the content of SBOMs, and the presence of sub-optimal and non-standardized tools, especially for what concerns the support for handling vulnerabilities.

Chaora et al. (2023) performed an ethnographic study analyzing discourse from public meetings held around SBOMs. The discourse occurred from September 20, 2022, to March 20, 2023, during SBOM community engagement meetings facilitated by the Cybersecurity & Infrastructure Security Agency (CISA) in the United States Department of Homeland Security. To promote SBOM adoption, the authors foresee regulations and a public-recognized agency that audits and validates SBOMs submitted to a common infrastructure. This infrastructure could restrict information on SBOM components to the authorized stakeholders and monitor vulnerabilities within them. The authors also foresee that SBOMs may face other challenges, such as supervising artificial intelligence.

[2] A direct dependency indicates that a component X depends on a component Y, while a transitive dependency indicates that a component X depends on a component Z because there is a component Y such that X depends on Y and Y depends on Z.

```
{
  "SPDXID" : "SPDXRef-DOCUMENT",
  "spdxVersion" : "SPDX-2.3",
  "creationInfo" : {
    "created" : "2023-12-31T18:48:20Z",
    "creators" : [ "Tool: spdx-maven-plugin" ],
    "licenseListVersion" : "3.22"
  },
  "name" : "Template",
  "dataLicense" : "CC0-1.0",
  "documentNamespace" :
  "http://spdx.org/spdxpackages/com.example_Template-1.0-SNAPSHOT",
  "packages" : [ {
    "SPDXID" : "SPDXRef-gnrtd0",
    "name" : "Template",
    "primaryPackagePurpose" : "LIBRARY",
    "versionInfo" : "1.0-SNAPSHOT"
  }, {
    "SPDXID" : "SPDXRef-gnrtd1",
    "description" : "Apache Log4j 1.2",
    "homepage" : "http://logging.apache.org/log4j/1.2/",
    "licenseDeclared" : "Apache-2.0",
    "name" : "Apache Log4j",
    "originator" : "Organization:Apache Software Foundation",
    "summary" : "Apache Log4j 1.2",
    "versionInfo" : "1.2.17"
  } ],
  "relationships" : [ {
    "spdxElementId" : "SPDXRef-DOCUMENT",
    "relationshipType" : "DESCRIBES",
    "relatedSpdxElement" : "SPDXRef-gnrtd0"
  }, {
    "spdxElementId" : "SPDXRef-gnrtd0",
    "relationshipType" : "DYNAMIC_LINK",
    "relatedSpdxElement" : "SPDXRef-gnrtd1"
  } ]
}
```


(a) SPDX SBOM.


```
{
  "bomFormat" : "CycloneDX",
  "specVersion" : "1.4",
  "serialNumber" : "urn:uuid:f8b8ab5d-0c68-47d9-82fe-65df75d30544",
  "version" : 1,
  "metadata" : {
    "timestamp" : "2023-12-31T17:46:04Z",
    "tools" : [ {
        "vendor" : "OWASP Foundation",
        "name" : "CycloneDX Maven plugin",
        "version" : "2.7.9"
    ],
    "component" : {
      "group" : "com.example",
      "name" : "Template",
      "version" : "1.0-SNAPSHOT",
      "type" : "library",
      "bom-ref" : "pkg:maven/com.example/Template@1.0-SNAPSHOT?type=jar"
    }
  },
  "components" : [ {
      "publisher" : "Apache Software Foundation",
      "name" : "log4j",
      "version" : "1.2.17",
      "licenses" : [ { "license" : { "id" : "Apache-2.0" } }
      ],
      "type" : "library",
      "bom-ref" : "pkg:maven/log4j/log4j@1.2.17?type=jar" }
  ],
  "dependencies" : [ {
      "ref" : "pkg:maven/com.example/Template@1.0-SNAPSHOT?type=jar",
      "dependsOn" : [ "pkg:maven/log4j/log4j@1.2.17?type=jar" ]
    }, {
      "ref" : "pkg:maven/log4j/log4j@1.2.17?type=jar",
      "dependsOn" : [ ]
    } ]
}
```


(b) CycloneDX SBOM.

Fig. 2. Examples of SBOMs generated using the SPDX and CycloneDX plugins for Maven.

Stalnaker et al. (2024) conducted a study combining surveys and interviews and aimed to investigate the challenges behind SBOM adoption for five different groups of stakeholders (practitioners familiar with SBOMs, contributors to critical open source projects, experts of Artificial Intelligence and Machine Learning, developers of cyber–physical systems, and legal experts), and the ways such challenges could be circumvented. The results of their study, which involved 138 participants, identified 12 different challenges and possible solutions to address them. Important challenges are related to the need to have multi-dimensional SBOM specifications, improve SBOM-related tools, aid SBOM verification/consumption, and favor SBOM adoption.

Kloeg et al. (2024) analyzed business stakeholder groups directly involved in SBOM production and consumption to determine which groups can drive or inhibit SBOM adoption and why. Through semi-structured interviews, the authors observed that system integrators and software vendors had the highest SBOM adoption potential, whereas business-to-business customers and individual developers had the lowest motivation.

Our study looks at the adoption of SBOMs from a different perspective than those by Xia et al. (2023), Chaora et al. (2023), Stalnaker et al. (2024), and Kloeg et al. (2024)—*i.e.,* we study the adoption of SBOM generation tools by open-source software projects, including produced SBOMs, rather than interviewing and surveying developers. Therefore, our study complements these studies and aims to increase our body knowledge on SBOM adoption.

Balliu et al. (2023) compared six state-of-the-art tools that generate CycloneDX SBOMs for Java Maven projects and discovered that these tools capture different sets of dependencies, which might be incorrect or incomplete. The authors encourage new research that deepens the analysis of SBOMs with respect to different SBOM standards and programming languages. In this respect, our investigation takes into account both CycloneDX and SPDX standards and various programming languages (*i.e.,* Go, Java, JavaScript, PHP, Python) taking a different perspective (*i.e.,* we do not perform a comparative study among SBOM generation tools).

Torres-Arias et al. (2023) performed a preliminary empirical investigation on NTIA's minimum data fields. They analyzed SPDX SBOMs from an SBOM dataset that was artificially created through different SBOM generation tools. The authors found that about one percent of the analyzed SBOMs fully complied with NTIA's minimum data fields. Differently from this study, we did not limit our analysis to only SPDX SBOMs, *i.e.,* we also considered CycloneDX ones. Moreover, we analyzed SBOMs retrieved directly from the repositories for which they were produced (*i.e.,* we relied on SBOMs produced by repositories' owners rather than artificially producing them). Among other things, we also investigated both NTIA's minimum and recommended data fields.

As far as SBOM generation tools, Yu et al. (2024) assessed GitHub's dependency graph, Trivy, Syft, and Microsoft SBOM Tool. Yu et al. analyzed SBOMs generated from 7876 open-source projects across nine programming languages. Their results revealed that all SBOM generation tools suffer from inaccuracies in generating SBOMs. Among the causes of such inaccuracies, Yu et al. reported incomplete metadata parsing, limitations in resolving transitive dependencies, limited support for version constraints, and inconsistent package naming conventions and dependency definitions. The primary difference between this work and ours is that we investigated the adoption of SBOM generation tools among open-source projects hosted on GitHub and not the quality of generated SBOMs. This difference mainly arises from the varying goals of the two research contributions.

2.3. Empirical studies on software supply chains

Other relevant literature is not directly related to SBOM, yet it studies the challenges and risks arising from software supply chains. These risks can be, indeed, mitigated through the adoption of SBOMs.

Levy (2003) asserts that open-source software can be more difficult to secure than proprietary software because of its development environment, which is more complex to a large extent. For example, organizations like Linux package the software commonly accepting code from others and bundling it with other packages. The resulting software is then distributed through websites. This complex web of interrelated parties creates several attack points in the software supply chain. The author highlights the need for technologies to secure supply chains of open-source software.

To enable sharing among industry practitioners having practical experiences and challenges with software supply chain security, Enck and Williams (2022) conduct two industry and one government software

summits (for a total of 30 organizations across the three summits). The result of these summits is a list of five software supply chain security challenges. Among these five top challenges, there is *leveraging SBOM for security*. While participants had widely divergent opinions, they agreed that efforts at establishing standards and requirements for SBOMs have the potential to lay the groundwork for innovative security enhancements.

Hartmann and Trew (2008) propose the concept of *Context Variability* to model multiple product lines and to support software supply chains. In detail, this model can facilitate the process of merging feature models during staged configuration where variability originates from different product types, geographic regions, and customers. Preliminary experiments have been conducted to assess the proposed model.

Lamb and Zacchiroli (2022) examine Reproducible-Builds (R-B) from the perspective of software professionals. R-B is an approach that can determine whether generated binaries correspond to a given source code. The main idea is that if we are confident that building a given source tree always generates bit-for-bit identical results, we can trust these artifacts. The results of their study indicate that uncontrolled build inputs occur when tool chains allow the build process to be affected by the surrounding environment. Similarly to Lamb and Zacchiroli (2022) and Vu et al. (2020) propose an approach, motivated by the intuition behind R-B, to detect injected code in typosquatting and hijacked packages. In particular, their approach detects code injected into software packages by comparing their distributed artifacts (*PyPI*) with the source code repository (GitHub). The approach has been preliminarily evaluated on two datasets of known malicious packages and the top ten most downloaded packages. Such an approach captures the known malicious packages and reduces the review effort for distributed software releases by 97%.

2.4. Empirical studies on software project inter-dependencies

There are also studies related to analyzing dependencies in the software ecosystem and their evolution. For example, Bavota et al. (2015) observe that the developers of the Apache Java ecosystem try to lower the impact that dependency upgrades would have when triggered by major releases and critical bug fixes. As for dependencies in R and npm ecosystems, Bogart et al. (2015) show that developers do not use systematic approaches to cope with changes. Later, Zahan et al. (2022) analyzed the metadata of 1.63 million JavaScript npm packages and proposed six alerts of security weaknesses (*e.g.,* install scripts and maintainer accounts associated with an expired email domain) in a software supply chain. Zahan et al. identified 11 malicious packages from the install scripts signal and found 2818 maintainer email addresses associated with expired domains (allowing the hijacking of 8494 packages). Zahan et al. also obtained qualitative feedback on link signals through a survey responded to by 470 npm package developers. One of the most important results of the survey is that the developers would want to be notified about weak link signals before using third-party packages.

3. Study design

The **goal** of our exploratory mining study is to analyze the adoption of SBOMs on open-source projects with the **purpose** of understanding *(i)* to what extent SBOMs are adopted; *(ii)* what is their adoption trend; *(iii)* how SBOMs are distributed; *(iv)* with which frequency SBOMs made available in software repositories get updated; *(v)* to what extent they comply with the standards; and *(vi)* what pieces of information SBOMs report, including NTIA's minimum and recommended data fields (NTIA, 2021g). The **perspective** is that of: *(i)* software producers interested in acquiring information on how SBOMs should be used to get the best from them; *(ii)* software consumers, such as end-users or those who commission the software, interested in understanding the weaknesses of consumed SBOMs; and *(iii)* researchers interested in fostering the spread of SBOMs and their correct use. The **context** is represented by open-source software projects available on GitHub and depending on SBOM generation tools owned by SPDX and CycloneDX.

3.1. Research questions

Based on the above-mentioned study goal, we formulated and investigated the following six Research Questions (RQs):

RQ1. *Are SBOMs Adopted by Open-source Software Projects?*

This RQ aims to understand whether open-source software projects adopt SBOMs by looking at projects that use SBOM generation tools based on the SPDX and CycloneDX standards.

RQ2. *What is the Adoption Trend of SBOMs by Open-source Software Projects and their Owners?*

With this RQ, we focus on the adoption trend of SBOMs, by looking at certain events of interest (*e.g.,* President Biden's Executive Order 14028 United States Government, 2021) that might increase the adoption of SBOMs by both open-source software projects and their owners.

RQ3. *How do Open-source Projects Distribute SBOMs?*

Here, we start investigating how SBOMs are managed by open-source software projects and, more specifically, how SBOMs are distributed, namely where SBOMs are placed (*e.g.,* under version control or just in public release versions).

RQ4. *How Frequently SBOMs Get Updated within Open-source Software Projects?*

This RQ contributes to understanding how SBOMs are managed in open-source software projects and, more specifically, with which frequency developers update the SBOMs they make available in these projects.

RQ5. *To What Extent do SBOMs Comply with the Standards?*

This RQ aims to verify whether SBOMs comply with the versions of the SPDX and CycloneDX standards they declare to be compliant with since uncompliant SBOMs might lead to failures during their ingestion by SBOM consumption tools.

RQ6. *What are the Pieces of Information Reported by the Found SBOMs?*

With this RQ we want to understand whether SBOMs comply with NTIA's minimum and recommended data fields (United States Department of Commerce, 2021) and what other information they report (*e.g.,* whether or not vulnerability information is included).

3.2. Study context and planning

The context of our study consists of open-source software projects available on GitHub—which is currently the most popular version-control and software-development-hosting platform for both personal and professional use (Stack Overflow, 2022)—and that depends on SBOM generation tools owned by SPDX or CycloneDX according to *GitHub Dependency Graph* (GitHub, 2023a). This tool allows getting all the dependencies and dependents of a software repository—*i.e.,* the repositories a given repository depends on and the repositories that depend on it, respectively. The information about the dependencies/dependents of a repository is automatically inferred from the manifest and lock files—*i.e.,* files specifying project dependencies such as `pom.xml` for Maven or `.yaml` for GitHub Actions—of the supported package managers, taking into account the default branch of that repository. Developers may also explicitly submit dependencies to GitHub Dependency Graph through the *Dependency Submission API* (beta).

As highlighted by Kalliamvakou et al. (2014), the analysis of data retrieved from GitHub can lead to conclusions that are not representative of the context that is intended to be investigated. To deal with such an issue, we excluded those repositories that:

- Were archived, and whose last push date was more than one month before the query date (March 5, 2023). The rationale was to avoid selecting inactive software projects (Kalliamvakou et al., 2014).
- Were forks. This was to avoid selecting duplicate software projects (Kalliamvakou et al., 2014).
- Had a number of commits less than 100. The rationale here was to avoid selecting software projects with a commit history too short and to mitigate the risk of selecting personal/toy projects (Kalliamvakou et al., 2014).
- Were not related to software development. This was to discard tutorials, books, students' assignments, and researchers' papers (Kalliamvakou et al., 2014). To do so, we searched within the name of the repositories and their owner for any instance of the terms `paper`, `book`, `tutorial`, `learn`, `bootcamp`, `sample`, `example`, `demo`, `test`, `mirror`, `copy`, `community`, `docs`, `theme`, and `template`. Then, we manually checked the suitability of the matched repositories and decided whether or not to discard them.

General information regarding the repositories (*e.g.,* number of commits and stars) and their owners was collected through *PyGithub* (PyGithub, 2023), a Python library that leverages *GitHub API v3*. As for commit-level information (*e.g.,* file modification dates and messages), it was retrieved through *PyDriller* (Spadini et al., 2018).

3.3. Data analysis

In this section, we describe our data analysis per RQ.

3.3.1. RQ1 (Are SBOMs Adopted by Open-source Software Projects?)

To identify the software projects that adopted SBOMs, we first selected SBOM generation tools among those whose software repository was available on GitHub and whose owner was SPDX or CycloneDX. Then, for each of the selected SBOM generation tools, we retrieved the list of repositories depending on them by leveraging GitHub Dependency Graph—this list of repositories is that of open-source software projects that adopted SBOMs.

SBOM generation tools can be integrated and used within software development pipelines to produce SBOMs and consume other SBOMs. Therefore, we needed to identify which software projects used these tools to generate their SBOMs rather than consuming other SBOMs. To do so, we relied on the classification of SBOM generation tools provided by SPDX (2023c) and CycloneDX (2023c). In particular, SBOM generation tools can be classified as:

- *Build-integration*, indicating tools to be integrated within build systems (*e.g.,* Maven) so that SBOMs can be automatically created when building software products. Note that a build-integration tool could even be executed as a command line tool within a CI/CD (Continuous Integration/Continuous Delivery) workflow.
- *GitHub Action*, when the tool is specifically conceived to work in a GitHub Actions CI/CD workflow. Such workflows are typically triggered when certain events (*e.g.,* a push on the main branch) occur.
- *Library*, indicating modular components that allow developers to programmatically produce, consume, or transform SBOMs.

We assumed that software projects using SBOM generation tools integrated into build systems or GitHub Actions workflows produce SBOMs. As for SBOM generation tools classified as a library, to ensure that they were used to produce SBOMs, we looked for the presence of SBOMs under version control or in the latest public release versions and excluded those projects that did not have any. To do so, we had to locally clone each repository and also download the latest public release version. Then, we searched within each file (including archive files such as TAR or ZIP) for the presence of the constituent parts of the SPDX and CycloneDX standards to determine whether or not that file was an SBOM. The list of the CycloneDX and SPDX constituent parts used to perform the search is shown in Table 2.

Table 2
List of CycloneDX and SPDX constituent parts.

Standard	Constituent part
CycloneDX	‘‘bomFormat’’:‘‘CycloneDX’’ xmlns="http://cyclonedx.org ‘‘$schema’’:"http://cyclonedx.org ‘bomFormat’:‘CycloneDX’ xmlns=’http://cyclonedx.org ‘$schema’:‘http://cyclonedx.org <vendor>CycloneDX</vendor> ‘‘$schema’’:"https://cyclonedx.org ‘$schema’:‘https://cyclonedx.org xmlns="https://cyclonedx.org xmlns=’https://cyclonedx.org
SPDX	SPDXVersion:SPDX- SPDXID:SPDXRef-DOCUMENT ‘‘SPDXID’’:‘‘SPDXRef-DOCUMENT’’ ‘‘spdxVersion’’:"SPDX- ‘‘SPDXID’’:"SPDXRef- ‘SPDXID’:‘SPDXRef-DOCUMENT’ ‘spdxVersion’:‘SPDX- ‘SPDXID’:‘SPDXRef-

We excluded projects that did not have files matching the aforementioned constituent parts.

For each project, based on the SBOM generation tools, we deduced the adopted SBOM standards. Then, we counted how many times each standard (SPDX or CycloneDX) was adopted by each software project, making a distinction based on whether the project was related to the domain of SBOMs or not. This was accomplished by looking for the following terms in the repository's name, description, and README file: `SBOM`, `CycloneDX`, `SPDX`, `Bill-of-Materials`, and `Supply-Chain`. If we found the occurrence of one of these terms for a project, we considered it to be related to the domain of SBOMs. This distinction helped us figure out how many of the projects that used SBOMs were directly related to the SBOM domain and, therefore, not just using SBOM generation tools but, rather, contributing to the SPDX or CycloneDX ecosystem.

3.3.2. RQ2 (What is the Adoption Trend of SBOMs by Open-source Software Projects and their Owners?)

For each repository, we collected information about the commit introducing an SBOM generation tool dependency by performing a *git log* operation on the manifest and lock files (*e.g.,* `pom.xml` for Maven or `.yaml` for GitHub Actions). This operation allowed us to (linearly) examine the commit history of these files, from the earliest to the oldest version, until we identified the commit introducing the SBOM generation tool dependency, along with its date.

We counted the number of adoptions of SBOM generation tools over time with respect to repositories, but also to owners, to control the influence that a few prolific owners might have compared to the whole sample size. We also took into account historically significant events related to SBOMs (*e.g.,* publication of standards or guidelines) to see whether they had an impact on the spread of SBOM generation tools.

We also categorized each repository based on the year in which it was created, through a 5-year interval scale: *2004–2008*, *2009–2013*, *2014–2018*, and *2019–2023*. We chose these intervals because they seemed to be a natural fit for our research purposes. According to these categories, we used boxplots showing the distributions of the dates when the dependencies towards the SBOM generating tools were added.

Finally, we assessed whether the introduction of a dependency towards an SBOM generation tool was explicitly mentioned in the message of the commit that introduced that dependency. To do so, we first searched into each commit message (taking into account only

the commits introducing SBOM generation tool dependencies) for the following terms: `SBOM`, `CycloneDX`, `SPDX`, `Bill-of-Materials`, and `Supply-Chain`. Then, each matched message was deemed an explicit mention of SBOM adoption, while non-matching messages were manually examined.

3.3.3. RQ3 (How do Open-source Projects Distribute SBOMs?)

For each dependency towards an SBOM generation tool, we looked for the presence of SBOMs under version control or in the latest public release version by searching for the constituents of SPDX and CycloneDX (similar to what was done for RQ1 when analyzing SBOM generation tools classified as a library). Also, for each project, we investigated whether the SBOMs appeared *(i)* under version control, *(ii)* in the latest public release version, *(iii)* on both version control and last public release, or *(iv)* nowhere.

Finally, as for the software projects using GitHub Actions to generate SBOMs, we checked what kind of event (*e.g.,* push, pull request) was specified in the CI/CD workflow to trigger the SBOM generation.

3.3.4. RQ4 (How Frequently SBOMs Get Updated within Open-source Software Projects?)

For each repository, we counted the commits that modified its SBOMs, starting from the commit in which they were created up to the last commit at the time of the analysis of this RQ (*i.e.,* April 1, 2023): the number of commits in this time interval was then converted in the time interval of a month to estimate SBOM changes per month.

For each repository that published SBOMs in the latest public release version (using the GitHub feature that allows for publishing a release version), we computed the percentage of public release versions containing SBOMs, either in the release archive or as a separate file.

Finally, for each commit in which an SBOM was modified, we checked whether it was associated with a *git tag* and whether that *git tag* was directly associated with a release of the repository—this was to understand whether the generation of a new SBOM occurred in correspondence with a new release. If there was no *git tag* associated with the commit that modified the SBOM, then the SBOM generation did not occur in correspondence with a new release.

3.3.5. RQ5 (To What Extent do SBOMs Comply with the Standards?)

A software project can distribute multiple SBOMs. This can happen when: *(i)* there are duplicated SBOMs (*i.e.,* same SBOM released in different file formats); *(ii)* SBOMs are targeted to different hardware/software configurations (*e.g.,* an SBOM is specific for the Windows operating system while another one is specific for Linux); or *(iii)* SBOMs are both under version control and available in public release versions. We discarded duplicated SBOMs from our analysis to avoid a misrepresentation of the phenomenon under investigation, while we kept those that were targeted to different hardware/software configurations since they contained different component information. As for SBOMs that were both under version control and available in public release versions, we kept the most up-to-date to reflect the current practice as much as possible. Then, to verify the validity of the distributed SBOMs for the SPDX and CycloneDX standards, taking into account the versions of these standards, we relied on the CycloneDX *sbom-utility* (v0.13.0) (CycloneDX, 2023e). This tool works with SBOMs in the JSON format; therefore, we had to translate the SBOMs that are not compliant with this format. In particular, to translate SBOMs from the XML (into the JSON) format, we used *cyclonedx-cli* (v0.25.0) (CycloneDX, 2023d). As for SBOMs released as plain text files, we used *SPDX Online Tool* (SPDX, 2023d).

The *sbom-utility* tool validates SBOMs against JSON schemata, one for each standard version, through the *validate* command. Table 3 reports the CycloneDX and SPDX schemata supported by this utility tool. To comply with a scheme, an SBOM must indicate the standard and the version thereof, along with all the required attributes for that standard version. It is worth mentioning that the required attributes for a given standard version do not necessarily correspond to the data fields deemed mandatory by other organizations (*e.g.,* NTIA's minimum data fields (United States Department of Commerce, 2021)). To specify the standard and version in an SBOM, CycloneDX provides the attributes *bomFormat* and *specVersion*, while SPDX provides the attributes *SPDXID* and *spdxVersion*. As for the other required attributes, they can vary among different versions of the same standard. For example, a CycloneDX SBOM with version 1.5 does not require any other attribute to be valid, while a CycloneDX SBOM with version 1.4 also requires the *version* attribute (which refers to the version of the SBOM, not to the standard version).

When an SBOM is not compliant with the schema of its standard version (*e.g.,* absence of required attributes or incorrect values for some attributes), the *sbom-utility* signals that the SBOM is not valid and reports the validity errors encountered.

Summing up, the output of the *sbom-utility* tool allowed us to check how many SBOMs were valid with respect to their standard versions, as well as know and quantify the kind of validity errors encountered.

3.3.6. RQ6 (What are the Pieces of Information Reported by the Found SBOMs?)

To identify which information is reported in CycloneDX and SPDX SBOMs, we parsed the JSON files of the SBOMs retrieved in the software projects—these are the same SBOMs studied in RQ5—and then extracted the (JSON) attributes along with their values. To map NTIA's minimum data fields (United States Department of Commerce, 2021) with the extracted attributes, we leveraged NTIA's mapping with respect to the SPDX and CycloneDX standards (NTIA, 2021g). Then, we counted how many SBOMs, projects, and owners contained each data field. As for NTIA's recommended data fields (United States Department of Commerce, 2021), we mapped each of these data fields with the remaining attributes and counted the number of times each data field (and its value) appeared, along with the number of SBOMs, projects, and owners involved.

We deepened the study of the *other unique identifiers* recommended data field by looking at which component or SBOM identifiers (*e.g.,* purl, CPE, *etc.*.) were used and to what extent (*i.e.,* how many SBOMs, projects, and owners were involved). Similarly, we deepened the study of the *component hash* recommended data field. Specifically, we looked at which hashing algorithms were used and to what extent. Moreover, we verified whether the same hash value (for a given hashing algorithm) was associated with different components.

To better analyze the dependency information reported in SBOMs, we built dependency graphs. To this aim, for each SBOM, we first identified the root node (*i.e.,* the node corresponding to the software product for which the SBOM was created) and then identified the direct and transitive dependencies from the root node. We generated the dependency graphs by using *NetworkX* (NetworkX, 2023), a Python package for creating, manipulating, and studying complex networks. We finally counted the number of SBOMs, projects, and owners where direct and transitive dependencies appeared.

4. Results

In this section, we present the results organized by RQ. A replication package containing the raw data regarding the software projects studied is available on the web (Nocera et al., 2024b).

4.1. RQ1: Are SBOMs Adopted by Open-source Software Projects?

Table 4 reports the standard used by each SBOM generation tool, its classification (*i.e.,* Build-integration, GitHub Action, or Library), the supported language, and the number of dependent software repositories we retrieved through the GitHub Dependency Graph. The majority of the dependent software repositories were associated with build-integration SBOM generation tools (*i.e.,* 166 occurrences in total, of

Table 3
List of CycloneDX and SPDX schemas supported by *sbom-utility* (CycloneDX, 2023e).

Standard	Version	Schema URL
CycloneDX	1.5	https://raw.githubusercontent.com/CycloneDX/specification/master/schema/bom-1.5.schema.json
	1.4	https://raw.githubusercontent.com/CycloneDX/specification/master/schema/bom-1.4.schema.json
	1.3	https://raw.githubusercontent.com/CycloneDX/specification/master/schema/bom-1.3.schema.json
	1.2	https://raw.githubusercontent.com/CycloneDX/specification/master/schema/bom-1.2.schema.json
SPDX	2.3	https://raw.githubusercontent.com/spdx/spdx-spec/development/v2.3/schemas/spdx-schema.json
	2.3.1	https://raw.githubusercontent.com/spdx/spdx-spec/development/v2.3.1/schemas/spdx-schema.json
	2.2.2	https://raw.githubusercontent.com/spdx/spdx-spec/v2.2.2/schemas/spdx-schema.json
	2.2.1	https://raw.githubusercontent.com/spdx/spdx-spec/v2.2.1/schemas/spdx-schema.json

Table 4
SBOM generation tools considered in our study.

Standard	Tool classification	Tool name	Language	# Dependents
CycloneDX	Build-integration	cyclonedx-cocoapods	Swift, Objective-C	0
		cyclonedx-conan	C, C++	0
		cyclonedx-dotnet	.NET	0
		cyclonedx-gomod	Go	2
		cyclonedx-gradle-plugin	Java	0
		cyclonedx-maven-plugin	Java	74
		cyclonedx-node-module	JavaScript	7
		cyclonedx-node-npm	JavaScript	3
		cyclonedx-php-composer	PHP	5
		cyclonedx-python	Python	64
		cyclonedx-ruby-gem	Ruby	0
		cyclonedx-rust-cargo	Rust	0
		cyclonedx-webpack-plugin	JavaScript	3
		Total		*158*
	GitHub Action	cdxgen-action	SourceImages, ContainerImages	0
		gh-cocoapods-generate-sbom	Swift, ObjectiveC	0
		gh-dotnet-generate-sbom	.NET	0
		gh-gomod-generate-sbom	Go	13
		gh-node-module-generatebom	JavaScript	2
		gh-php-composer-generate-sbom	PHP	1
		gh-python-generate-sbom	Python	1
		Total		*17*
	Library	cyclonedx-core-java	Java	2
		cyclonedx-dotnet-library	.NET	0
		cyclonedx-go	Go	4
		cyclonedx-javascript-library	JavaScript	0
		cyclonedx-node-pnpm	JavaScript	0
		cyclonedx-node-yarn	JavaScript	0
		cyclonedx-php-library	PHP	0
		cyclonedx-python-lib	Python	1
		Total		*7*
SPDX	Build-integration	spdx-maven-plugin	Java	8
		Total		*8*
	Library	Spdx-Java-Library	Java	1
		spdx-tools-js	JavaScript	0
		tools	Java	1
		tools-golang	Go	4
		tools-java	Java	0
		tools-python	Python	1
		Total		*7*

which 8 for SPDX and 158 for CycloneDX), followed by those classified as GitHub Action (*i.e.,* 17 occurrences, all for CycloneDX). It is worth mentioning that 11 repositories depended on more than one SBOM generation tool; therefore, the total number of distinct repositories that relied on an SBOM generation tool was 186.

Table 5 reports some descriptive statistics—*i.e.,* mean, Standard Deviation (SD), minimum (min), median, and maximum (max)—about the dependent repositories we studied (*e.g.,* their number of stars). The values of these descriptive statistics suggest that the sample is quite heterogeneous in terms of the number of commits, stars, subscribers, contributors, and forks. Furthermore, the number of commits and contributors is greater than 579.5, with a median value of 4.5. This indicates a noticeable collaborative development activity in these software projects.

Table 5
Some descriptive statistics about the projects dependent on SBOM tools.

Metric	Mean	SD	Min	Median	Max
# Commits	2463.72	7367.49	104	579.5	63,328
# Stars	937.14	3674.42	0	3	35,126
# Subscribers	45.41	186.28	0	3	2,056
# Contributors	35.85	81	1	4.5	364
# Forks	431.7	2273.05	0	1	26,661

Table 6 shows information on the adoption of the SBOM standards, taking into account whether or not they were related to the SBOM

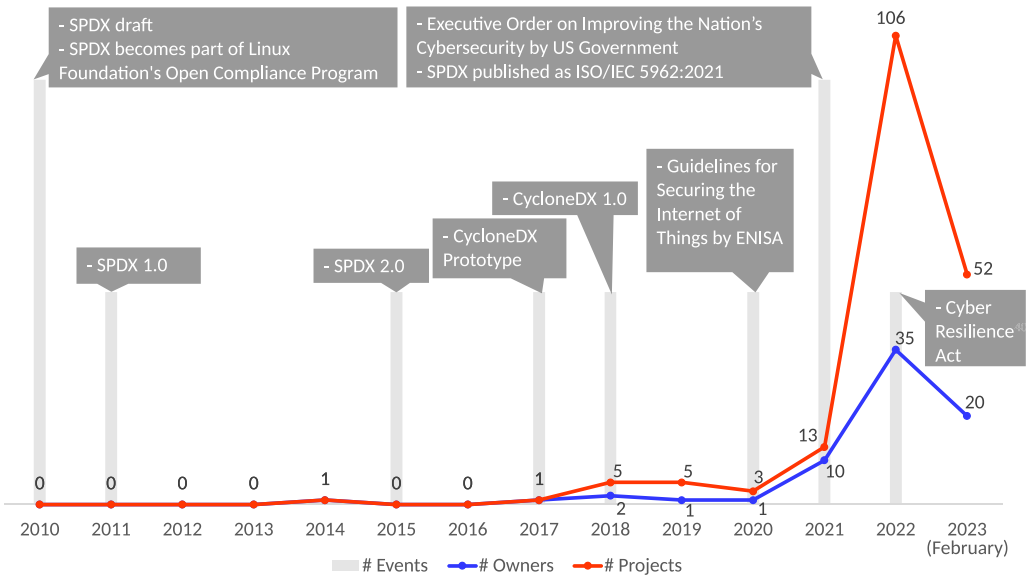


Fig. 3. SBOM generation tools adoption over time by the studied projects.

Table 6
Adoption of SPDX and CycloneDX among the studied projects.

Domain	Standard	# Projects
SBOM-related	Only SPDX	8
	Only CycloneDX	22
	Both SPDX and CycloneDX	1
SBOM-unrelated	Only SPDX	3
	Only CycloneDX	151
	Both SPDX and CycloneDX	1
Overall	Only SPDX	11
	Only CycloneDX	173
	Both SPDX and CycloneDX	2

domain. CycloneDX is the most widely used standard, accounting for 94% of the cases (*i.e.,* $(173 + 2)/186 * 100$). Two software projects adopted both the CycloneDX and SPDX standards. Also, 31 (17%) software projects using SBOM generation tools are related to the SBOM domain.

RQ1 summary: SBOMs are not widely adopted in open-source projects. The most used standard is CycloneDX, and the preferred way to generate SBOMs is through build-integration tools.

4.2. RQ2: What is the Adoption Trend of SBOMs by Open-source Software Projects and their Owners?

Fig. 3 depicts when, over the evolution histories, repositories and owners started adopting SBOM generation tools. In particular, the blue and orange lines highlight, year by year, how many new owners and repositories started adopting SBOM generation tools, respectively. The gray bars indicate major events involving SBOMs and software supply chain practices (*e.g.,* the conception and release of the SPDX and CycloneDX standards (SPDX, 2024a; CycloneDX, 2024a), President Biden's Executive Order 14028 (United States Government, 2021), *etc.*). Based on the gathered data, SBOM generation tools have started to be adopted by open-source software projects hosted on GitHub since 2014. As shown in Table 7, initially these tools were mainly adopted by those projects whose owner was related to the SBOM domain (*e.g.,* SPDX and CycloneDX).

Table 7
Projects that have adopted SBOM generation tools till 2020.

Owner	Project	SBOM Adoption Date
spdx	spdx-maven-plugin	2014-01-13
CycloneDX	cyclonedx-maven-plugin	2017-06-24
spdx	LicenseListPublisher	2018-01-29
stevespringett	Alpine	2018-05-02
CycloneDX	cyclonedx-core-java	2018-05-30
DependencyTrack	dependency-track	2018-09-18
DependencyTrack	hyades-apiserver	2018-09-18
ottlinger	mailclena	2019-05-02
spdx	Spdx-Java-Library	2019-06-09
DependencyTrack	frontend	2019-11-01
CycloneDX	cyclonedx-php-composer	2019-11-19
spdx	spdx-java-rdf-store	2019-12-12
spdx	tools-java	2020-01-06
spdx	spdx-java-jackson-store	2020-01-18
MO-Movia	licit	2020-10-21

Since 2021, the interest in SBOM generation tools has increased, coinciding with *(i)* President Biden's Executive Order 14028 (United States Government, 2021) and *(ii)* the publication of the SPDX standard as ISO/IEC 5962:2021 (ISO, 2021). A further boost to the adoption of SBOM generation tools seems to be due to the Cyber Resilience Act proposed in 2022 (European Commission, 2022). Some owners were particularly prolific in the adoption of SBOMs for their repositories (blue line of Fig. 3). However, it is worthwhile noting an increasing number of new authors are utilizing SBOM generation tools (orange line of Fig. 3).

Fig. 4 shows the distribution of SBOM adoption dates by software projects, grouping them by the year in which they were created. In general, projects created between 2014 and 2018 are those that first adopted SBOM generation tools (see the mean and median values). Indeed, these projects are related to the SBOM domain (see Table 7). Next, we find those projects created between 2019 and 2023, those between 2009 and 2013, and, lastly, those between 2004 and 2008.

Finally, SBOMs were explicitly mentioned in the commit messages that introduced the dependencies towards SBOM generation tools for 90 out of 186 repositories (*i.e.,* in 48% of the studied software projects).

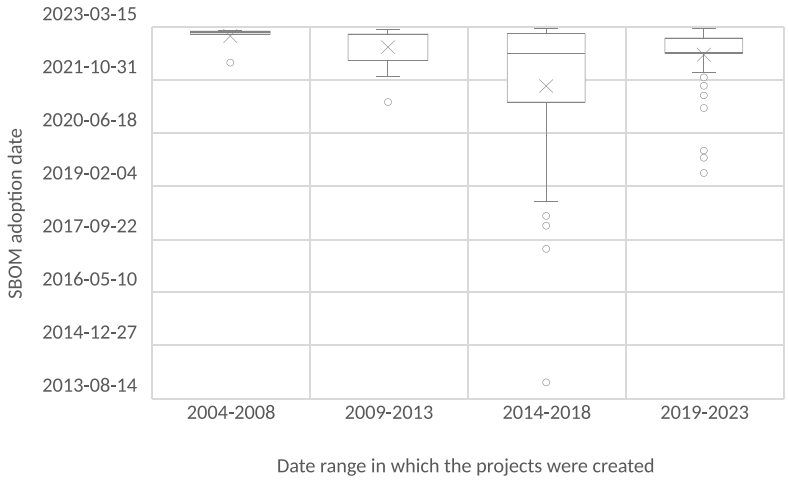


Fig. 4. Boxplots showing the distribution of SBOM adoption dates.

RQ2 summary: SBOMs started to be adopted by open-source software projects whose owners relate to the SBOM domain; indeed, the projects that first adopted SBOMs were those created between 2014 and 2018. Since 2021, there has been a growing interest in SBOMs, even from those not related to the SBOM domain. Although we do not have causality evidence, this seems to coincide with both President Biden's Executive Order 14028 and the publication of the SPDX standard as ISO/IEC 5962:2021. The Cyber Resilience Act proposed in 2022 seems to increase interest in SBOMs.

4.3. RQ3: How do Open-source Projects Distribute SBOMs?

Out of 186 software projects, 55 kept SBOMs under version control only, 14 distributed SBOMs in public release versions only, and 16 used both strategies to distribute SBOMs (*i.e.,* we found SBOMs both under version control and in public release versions). Summing up, SBOMs are available under version control or published in public release versions in 46% of the software projects analyzed. For the remaining 101 software projects using SBOM generation tools, we found no SBOM. This suggests that either SBOMs are distributed differently or those interested in SBOMs have to generate SBOMs on their own (*e.g.,* by building the software product).

As for the software projects using GitHub actions to generate SBOMs, we observed that the generation of SBOMs is triggered by a push and pull request in 88% and 59% of the projects, respectively. Only one project generated SBOMs during the release activity. In other words, in the projects that generate SBOMs by relying on GitHub actions, SBOMs tend to be continuously generated.

Very few software projects distributed multiple SBOMs beyond simply including them in both version control and public release versions. That is, 3% of the software projects provided SBOMs in different file formats (*i.e.,* JSON and XML). Another 3% of software projects generated distinct SBOMs to reflect the components required to run/deploy the software according to specific software/hardware (*e.g.,* SBOM tailored for different operating systems, such as Windows or Linux).

RQ3 summary: In most projects, SBOMs are not versioned nor attached to public release versions. Hence, they are likely to be distributed differently. In the remaining cases, it appears that project owners prefer to keep SBOMs under version control rather than including them in public release versions. Projects that leverage GitHub actions to generate SBOMs, in most cases, do so at every push or pull request.

Table 8
Number of analyzed SPDX and CycloneDX SBOMs, along with the number of projects and owners distributing them.

Standard	# SBOMs	# Projects	# Owners
CycloneDX	89	78	16
SPDX	30	6	6
Total	*119*	*84*	*22*

4.4. RQ4: How Frequently SBOMs Get Updated within Open-source Software Projects?

Considering those software projects with SBOMs under version control, we observed that the median number of changes per month is 0.96. However, this number is highly variable, as indicated by the SD (1.74), min (0.03), and max (9.72) values. Overall, this suggests periodic SBOM updates, which could be associated with public release versions.

Limiting the attention to software projects that distribute SBOMs in public release versions, SBOMs appear on average in 61% of all public releases of a given software project (and 51% on median). If we consider only the release versions starting from the SBOM adoption date, only three software projects (out of all 30 software projects having SBOMs in their release versions) have some release versions without SBOMs. In particular, one software project has one release version without SBOMs, another has two, and yet another has six.

Finally, we observed that only two public release versions (out of all public release versions publishing SBOMs) are tagged with a commit that modified SBOMs in the repository. There are eight more tags representing commits in which SBOMs were modified, but no release version is associated with them. Due to the low number of matches, the generation of a new SBOM did not usually occur in correspondence with a new release.

RQ4 summary: SBOMs under version control are usually updated every month, while those distributed through release versions are usually updated at every release. In this respect, publishing new release versions does not occur at the time of SBOM generation, and release versions are typically not tagged with commits that modify SBOMs.

4.5. RQ5: To What Extent do SBOMs Comply with the Standards?

Table 8 shows the number of SPDX and CycloneDX SBOMs we analyzed, along with the number of software projects and owners distributing them. We analyzed 119 SBOMs—89 referred to the CycloneDX

Table 9
Number of selected SPDX and CycloneDX SBOMs valid with respect to their SBOM standard version.

Standard	Schema version	Valid	Not valid
CycloneDX	1.2	1	0
	1.3	0	1
	1.4	81	3
	1.5	3	0
SPDX	2.2	1	0
	2.3	19	10

standard, while 30 referred to SPDX—from a total of 84 projects and 22 owners, respectively. We had to discard an SPDX SBOM, released as a plain text file, because the SPDX Online Tool failed to translate into the JSON format.

Table 9 shows the number of SBOMs (valid or not) with respect to the version of the SBOM standard they had to comply with. Out of 89 CycloneDX SBOMs, 85 (96%) were valid. These invalid CycloneDX SBOMs belonged to different software projects. As for SPDX, the number of valid SBOMs was equal to 20 (67%) out of 30 SPDX SBOMs. The invalid SPDX SBOMs were all distributed with the same software project.

Table 10 shows the errors resulting from the SBOMs not being valid with respect to their standard version. As for CycloneDX, most errors relate to URL (Uniform Resource Locator) formatting in the *externalReferences* attribute. This attribute allows specifying external references, which provide a way to document systems, sites, and information relevant to the objects (*e.g.,* components, services, or even SBOMs) with which they are associated. An external reference can be defined through a URL and accept any URL scheme (CycloneDX, 2023a). The errors we found are due to having reported incorrect URLs (*e.g.,* https://github.com:ethlo/itu or git://github.com:appendium/flatpack.git/flatpack). The other two types of validity errors in CycloneDX involve pattern mismatch for the value of a *serialNumber* attribute and the absence of a *version* attribute. The *serialNumber* attribute indicates the unique serial number of the SBOM, which must conform to RFC-4122.[3] As for the version attribute, its value indicates the version number of the SBOM: It is one by default and should be incremented by one whenever the existing SBOM is modified.

As for SPDX, the error type returned by the *sbom-utility* tool was always *unknown*—*i.e.,* the tool could not comprehend the reasons why these SBOMs were not valid. Therefore, we manually checked these SPDX SBOMs to understand the reasons for unknown errors. We found that these SBOMs contained four attributes (*predicate*, *statement*, *envelope*, and *bundle*) not belonging to the SPDX standard, and one of them (*i.e., predicate*) contained the SPDX SBOM data. This is because these SBOMs were compliant with the *in-toto* attestation framework, which allows securing the integrity of software supply chains (in-toto, 2023b; Torres-Arias et al., 2019).

The above-mentioned validity errors might negatively affect tools consuming SBOMs depending on the kind of errors, SBOM use case, and/or SBOM consumption tool. For instance, let us consider the SPDX SBOMs with unknown validity errors we found when executing *sbom-utility* to check for the presence of vulnerabilities. The tool failed with the following error message: `[ERROR] format not supported: unknown format`. On the other hand, this tool would not fail for the CycloneDX SBOMs having the validity errors reported above. However, SBOM consumption tools supporting other SBOM use cases might fail. The causes behind the validity errors we found can be different. For instance, the unknown validity errors are due to the use of the in-toto attestation framework. As for the incorrect values of attributes (*i.e.,* incorrect formats and patterns), this could originate from the data sources that the SBOM generation tools use to produce SBOMs.

RQ5 summary: A limited fraction of SBOMs exhibit validity errors with respect to their SBOM standard version. These errors concern mismatched patterns for the values specified for some attributes and invalid SBOM structure. These validity errors might compromise SBOM consumption.

4.6. RQ6: What are the Pieces of Information Reported by the Found SBOMs?

Table 11 reports, for each NTIA's minimum data field, the number (and percentage) of SBOMs reporting it, along with the number (and percentage) of projects and owners involved. We report the number of SBOMs, projects, and owners containing *non-asserted* data fields—*i.e.,* those for which the value is not provided (*e.g., none* or *null* value)—in round brackets. The analyzed SPDX SBOMs reported all NTIA's minimum data fields at least once but the *supplier name* and *dependency relationship*, appearing in only one SBOM. Therefore, only 3% ($1/30 * 100$) of the analyzed SPDX SBOMs reported all NTIA's minimum data fields. As for CycloneDX SBOMs, the supplier name appeared at least once in only seven SBOMs, resulting in the least appearing data field across CycloneDX SBOMs. These seven CycloneDX SBOMs reported at least once all NTIA's minimum data fields. This is to say that only 8% ($7/89 * 100$) of the analyzed CycloneDX SBOMs reported all NTIA's minimum data fields. The only data field that all CycloneDX SBOMs reported at least once is *other unique identifiers*. Only two and three CycloneDX SBOMs did not report any occurrence of the *author of SBOM data* and *timestamp* data fields, respectively. As for *dependency relationships*, this data field is reported at least once in 78 CycloneDX SBOMs (out of 89). The *component name* and *component version* data fields are reported at least once in 80 CycloneDX SBOMs. This indicates that nine CycloneDX SBOMs did not report any component. The presence of SBOMs that do not report components highlights an improper use of SBOMs. However, if we consider the number of owners, all of them reported the *component name* and *component version* data fields in some of their SBOMs.

Table 12 shows the different kinds of *other unique identifiers* used to identify software components and even SBOMs, along with the number (and percentage) of SBOMs, projects, and owners reporting them. As for CycloneDX SBOMs, *purl* is the most reported identifier for software components, appearing in 90% ($80/89 * 100$) of them. Only one CycloneDX SBOM reports two CPE identifiers. As for SPDX SBOMs, the most introduced identifier for software components is SPDX ID, which appears in all of them. To follow, we find *purl* and CPE, appearing in 97% ($29/30 * 100$) and 93% ($28/30 * 100$) of SPDX SBOMs. In addition, each SPDX SBOM declared an SPDX ID to identify the SBOM (which always assumes the value *SPDXRef-DOCUMENT*). Similarly, each SPDX SBOM provided a document-specific namespace as a URI (Uniform Resource Identifier). Also, all CycloneDX SBOMs have an instance of *UUID* (*Universally Unique IDentifier*) to identify SBOMs. UUID identifiers are also used to identify software components in 60% ($53/89 * 100$) of the Cyclonedx SBOMs.

Similarly to NTIA's minimum data fields, Table 13 reports, for each NTIA's recommended data field, the number (and percentage) of SBOMs reporting it, along with the number (and percentage) of projects and owners involved. Only 29% ($26/89 * 100$) of the CycloneDX SBOMs report at least once a component hash; however, the percentage of CycloneDX owners reporting component hash is high, namely 81% ($13/16 * 100$). As for SPDX SBOMs, the percentages of SBOMs and owners reporting component hash at least once are similar, namely 63% ($19/30 * 100$) and 67% ($4/6 * 100$), respectively. Delving deeper

[3] The RFC-4122 specification defines a Uniform Resource Name (URN) namespace for UUIDs (Universally Unique IDentifier), which are 128 bits long and can guarantee uniqueness across space and time (Joinup, 2023).

Table 10
Validity errors for the analyzed SBOMs with respect to their standard version.

Standard	Error type	Attribute	Description	# Errors
CycloneDX	Format	externalReferences	Does not match format "iri-reference"	8
	Pattern	serialNumber	Does not match pattern "^urn:uuid:[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$"	1
	Required	version	Version is required	1
SPDX	Unknown	unknown	Unknown	11

Table 11
NTIA's minimum data fields discovered in the analyzed SBOMs (in round brackets, occurrences of non-asserted data fields).

Data field	Standard	SBOMs		Projects		Owners	
		#	%	#	%	#	%
Supplier name	CycloneDX	7	*8*	7	*9*	4	*25*
	SPDX	1 (1)	*3 (3)*	1 (1)	*17 (17)*	1 (1)	*17 (17)*
Component name	CycloneDX	80	*90*	70	*90*	16	*100*
	SPDX	30	*100*	6	*100*	6	*100*
Component version	CycloneDX	80 (1)	*90 (1)*	70 (1)	*90 (1)*	16 (1)	*100 (6)*
	SPDX	30	*100*	6	*100*	6	*100*
Other unique identifiers	CycloneDX	89	*100*	78	*100*	16	*100*
	SPDX	30	*100*	6	*100*	6	*100*
Dependency relationship	CycloneDX	78	*88*	68	*87*	14	*88*
	SPDX	1	*3*	1	*17*	1	*17*
Author of SBOM data	CycloneDX	87	*98*	77	*99*	16	*100*
	SPDX	30 (1)	*100 (3)*	6 (1)	*100 (17)*	6 (1)	*100 (17)*
Timestamp	CycloneDX	86	*97*	76	*97*	15	*94*
	SPDX	30	*100*	6	*100*	6	*100*

Table 12
Identifier information reported for *other unique identifiers* data field.

Standard	Identifier type	Target	SBOMs		Projects		Owners	
			#	%	#	%	#	%
CycloneDX	CPE	Software component	1	*1*	1	*1*	1	*6*
	purl	Software component	80	*90*	70	*90*	16	*100*
	UUID	Software component	53	*60*	53	*68*	3	*19*
		SBOM	89	*100*	78	*100*	16	*100*
SPDX	CPE	Software component	28	*93*	4	*67*	4	*67*
	SPDX ID	Software component	30	*100*	6	*100*	6	*100*
		SBOM	30	*100*	6	*100*	6	*100*
	purl	Software component	29	*97*	5	*83*	5	*83*
	URI	SBOM	30	*100*	6	*100*	6	*100*

Table 13
NTIA's recommended data fields discovered in the analyzed SBOMs (in round brackets, we report the number of non-asserted data fields).

Data field	Standard	SBOMs		Projects		Owners	
		#	%	#	%	#	%
Component hash	CycloneDX	26	*29*	16	*21*	13	*81*
	SPDX	19	*63*	4	*67*	4	*67*
Life cycle phase	CycloneDX	0	*0*	0	*0*	0	*0*
	SPDX	0	*0*	0	*0*	0	*0*
Other component relationships	CycloneDX	0	*0*	0	*0*	0	*0*
	SPDX	30	*100*	6	*100*	6	*100*
License information	CycloneDX	30	*34*	19	*24*	14	*88*
	SPDX	30 (30)	*100 (100)*	6 (6)	*100 (100)*	6 (6)	*100 (100)*

into the analysis of the component hash data field, Table 14 conveys information on the different kinds of hashing algorithms used. The most commonly used hashing algorithm is *SHA-256* for both SPDX and CycloneDX SBOMs. It is worth noting that multiple hashes can be provided for the same component. For instance, *SHA3-256* hashes are always reported along with *SHA3-384* and *SHA3-512* ones. This might come in handy when external sources/databases support different hash types. In this respect, we verified that, across all SBOMs, each component was identified by only one hash value for each different hashing algorithm. However, we found that the same hash value could map different components. More specifically, we found that this happened only once in CycloneDX SBOMs and involves the following two components:

- `pkg:maven/io.grpc/protoc-gen-grpc-java@1.53.0?classifier=osx-aarch_64&type=exe`
- `pkg:maven/io.grpc/protoc-gen-grpc-java@1.53.0?classifier=osx-x86_64&type=exe`

As shown above, these components have similar purl identifiers, differing only in one parameter specifying the kind of operative system architecture they are compliant with. In other words, these two purl identifiers refer to the same component. As for SPDX SBOMs, we found

Table 14
Component hash information reported in the SBOMs.

Standard	Hashing algorithm	SBOMs		Projects		Owners	
		#	%	#	%	#	%
CycloneDX	MD5	7	*8*	7	*9*	4	*25*
	SHA-1	8	*10*	8	*10*	5	*31*
	SHA-256	25	*28*	15	*19*	12	*75*
	SHA3-256	6	*7*	6	*8*	3	*19*
	SHA3-384	6	*7*	6	*8*	3	*19*
	SHA3-512	6	*7*	6	*8*	3	*19*
	SHA-384	7	*8*	7	*9*	4	*25*
	SHA-512	8	*9*	8	*10*	5	*31*
SPDX	SHA-1	1	*3*	1	*17*	1	*17*
	SHA-256	18	*60*	3	*50*	3	*50*

210 hash values mapping more than one component each, all associated with CPE identifiers. We verified whether these CPE identifiers mapped by the same hash values were similar to each other—for instance, what changes is the use of a different character (*e.g.,* an underscore instead of a dash) or the presence of a few extra words specifying something more about the component. These CPE identifiers refer to the same component. Having different CPE (or *purl*) identifiers referring to the same component might be an issue since external sources/databases might not know all CPE (or purl) identifiers for a given component. Given a larger number of cases using the same hash value mapping different components identified by CPE, we might infer that purl allows for a more robust representation of components than CPE.

As shown in Table 13, no SPDX or CycloneDX SBOM ever reported the *life cycle phase* data field; however, from the used SBOM generation tools, we can infer that the SBOM generation happens within- or post-building. As for the *other component relationship* data field, no occurrence was ever found in CycloneDX SBOMs, while all SPDX SBOMs reported some introductions. Table 15 shows the different kinds of relationships (different from dependency) specified in the SPDX SBOMs, along with a brief description according to the SPDX documentation (SPDX, 2023a)—note that *SPDXRef-A* and *SPDXRef-B* refer to generic components (or also files), while *SPDXRef-DOCUMENT* to the SBOM itself. The most common relationship different from dependency was *describes*, which appeared in 29 SPDX SBOMs, each with one occurrence. *Dynamic link* appeared in only two SPDX SBOMs. Both *Generates* and *Test case of* appeared in one SBOM only.

Considering the *license information* data field (see Table 13), we found at least one occurrence in 34% (30/89*100) of the analyzed CycloneDX SBOMs, while all SPDX SBOMs reported license information at least once.

Table 16 reports the number (and percentage) of SBOMs having direct and transitive dependencies within their dependency graphs, along with the number of projects and owners involved. First, the number of SBOMs describing the *primary component* (*i.e.,* the software product for which the SBOM was produced) in the sections reporting component information is 26 for CycloneDX and 30 for SPDX. Every node in the dependency graphs of these SBOMs is also described in the sections reporting component information. However, the opposite is not true. Namely, we found cases where a component was described in the section reporting component information but it did not appear as a node in the dependency graph. We can see that all the 23 CycloneDX SBOMs having direct dependencies also have some transitive dependencies; on the other hand, only 2 SPDX SBOMs have transitive dependencies. Each SPDX SBOM reported the primary component as a node in the dependency graph while only 23 CycloneDX SBOMs do it (see the number of SBOMs having direct dependencies). However, we observed that not all nodes were linked to the primary component nodes in the dependency graphs. In particular, we found a path between the primary component node and any other node in only 20 CycloneDX and 28 SPDX SBOMs, respectively.

Table 17 provides an overview of how frequently NTIA's minimum and recommended data fields were discovered in all analyzed SBOMs, without distinguishing per standard. The data fields are sorted from the least discovered to the most discovered in SBOMs. The only data field that is completely missing is *life cycle phase*, which may suggest that the considered SBOM generation tools do not provide support for generating this data field. *Supplier name* was discovered in only 7% (8/119 ∗ 100) of SBOMs, and, in one of them, its value might not be asserted. This means that very few SBOMs include supplier information.

The *other component relationships* was reported in 25% of SBOMs (30/119 ∗ 100), while *component hash* of 38% of them (45/119 ∗ 100). *License information* was discovered in 50% of SBOMs (60/119 ∗ 100), and in 25% (30/119 ∗ 100) possibly with non-asserted values. The remaining data fields appear in more than 90% of SBOMs. In detail, *dependency relationship* was discovered in 91% of SBOMs (108/119 ∗ 100), *component name* and *component version* in 92% (110/119 ∗ 100), *timestamp* in 97% (116/119 ∗ 100), and *author of SBOM data* in 98% (117/119 ∗ 100). The presence of one non-asserted instance of *component version* and *author of SBOM data* involves just one SBOM. *Other unique identifiers* was the only data field always discovered in the analyzed SBOMs.

As shown in Table 17, similar considerations can also be made for projects and owners on how frequently NTIA's minimum and recommended data fields were discovered in all analyzed SBOMs. This omission of information about NTIA's minimum and recommended data fields would hinder the pursuit of basic use cases (*e.g.,* handling software inventory or verifying license compliance) as well as advanced use cases (*e.g.,* tracking software provenance or pedigree) for the identified software projects.

Finally, no vulnerability information was reported in either CycloneDX or SPDX SBOMs. Providing information about vulnerabilities has been advocated in previous work (Stalnaker et al., 2024; Torres-Arias et al., 2023; Xia et al., 2023). Moreover, a recent study experimented with the usefulness of vulnerability-augmented SBOMs (Fucci et al., 2025a).

RQ6 summary: Only a few SBOMs report information on all NTIA's minimum data fields, with some not reporting any component information. SBOMs mostly use *purl* and then CPE to identify components, no occurrence of SWID tags has been ever found. As for NTIA's recommended fields, no SBOM reports information on all of them, as no information on the life cycle phase was ever reported. No vulnerability has ever been reported in any SBOM.

5. Discussion

In this section, we present and discuss the distilled main findings of our study. We conclude by discussing threats that might impact the validity of our results.

5.1. Main findings

Below, we summarize the main findings of our study and delineate implications for researchers, software producers, and software consumers. Due to the nature of our study (explorative), we also present possible future research directions taking into consideration our outcomes and those from past studies.

Finding #1: Based on projects' dependencies, a limited number of open-source software projects adopt SBOM generation tools. However, such a number is progressively increasing.

Table 15
Types of dependencies reported in the SPDX SBOMs.

Relationship type	SBOMs		Projects		Owners	
	#	%	#	%	#	%
Describes (SPDXRef-DOCUMENT describes SPDXRef-A)	29	*97*	5	*83*	5	*83*
Dynamic link (SPDXRef-A dynamically links to SPDXRef-B)	2	*7*	2	*33*	2	*33*
Generates (SPDXRef-A generates SPDXRef-B)	1	*3*	1	*17*	1	*17*
Test case of (SPDXRef-A is a test case used in testing SPDXRef-B)	1	*3*	1	*17*	1	*17*

Table 16
Number of direct and transitive dependencies reported in the SBOM files.

Dependency	Standard	SBOMs		Projects		Owners	
		#	%	#	%	#	%
Direct	CycloneDX	23	*26*	13	*17*	10	*63*
	SPDX	30	*100*	6	*100*	6	*100*
Transitive	CycloneDX	23	*26*	13	*17*	10	*63*
	SPDX	2	*7*	2	*33*	2	*33*

Table 17
NTIA's minimum and recommended data fields discovered in the analyzed SBOMs. In round brackets, we report the number of non-asserted data fields.

Data field	SBOMs		Projects		Owners	
	#	%	#	%	#	%
Life cycle phase	0	0	0	0	0	0
Supplier name	8 (1)	7 (1)	8 (1)	10 (1)	5 (1)	23 (5)
Other component relationships	30	25	6	7	6	27
Component hash	45	38	20	24	17	77
License information	60 (30)	50 (25)	25 (6)	30 (7)	20 (6)	91 (27)
Dependency relationship	108	91	74	88	20	91
Component name	110	92	76	90	22	100
Component version	110 (1)	92 (1)	76 (1)	90 (1)	22 (1)	100 (1)
Timestamp	116	97	82	98	21	95
Author of SBOM data	117 (1)	98 (1)	83 (1)	99 (1)	22 (1)	100 (5)
Other unique identifiers	119	100	84	100	22	100

Although we based our study on 35 SBOM generation tools officially released by SPDX or CycloneDX — the two major SBOM standards (Xia et al., 2023) —, we could identify only 186 open-source projects (see Section 3.2) adopting SBOMs. This suggests that SBOMs are still neglected by the open-source community, probably due to the lack of interest and awareness in the software supply chain and its security among **software producers** and **consumers**. The limited adoption of SBOMs can be considered consistent with what was reported by Xia et al. (2023). We conjecture that the low adoption of SBOMs in open-source projects might be due to the lack of incentives for generating SBOMs (Xia et al., 2023; Stalnaker et al., 2024; Kloeg et al., 2024). For example, open-source software projects may not be required to have an SBOM (Stalnaker et al., 2024), unlike software projects commercialized to the United States federal agencies (United States Government, 2021). However, open-source software projects may soon be pressured to supply an SBOM because commercialized software could depend on them (The Linux Foundation, 2022a). The adoption of SBOMs is expected to grow due to the demand of major players, including governmental organizations. **Software producers** cannot wait any longer and should begin to identify effective strategies for producing and distributing their SBOMs. We advise **software producers** to begin integrating SBOM generation into their development pipelines, anticipating possible regulations that could mandate SBOM compliance. For example, SBOM generation tools like *cyclonedx-maven-plugin* or *spdx-maven-plugin* can be integrated with minimal effort in projects already using Maven as a package manager.

Finding #2: The preferred way to generate SBOMs is through build-integration tools, followed by GitHub Action. The generation of SBOMs via libraries is less common than in the ways mentioned above.

As pointed out by Xia et al. (2023), one of the challenges restricting the adoption of SBOMs is the limited availability of suitable tools and the features offered by such tools. However, standard creators (*i.e.,* SPDX and CycloneDX) and forges (*e.g.,* GitHub) are working to make available to **software producers** convenient tools for SBOM generation. In particular, we observed a wider adoption of SBOMs via build-integration tools (*e.g.,* Maven) and CI/CD tools (*i.e.,* GitHub Action) compared to the adoption of SBOMs via libraries. This is probably due to the ease of use of both build-integration and CI/CD tools: the former allows SBOMs to be generated when building a software product while the latter when a certain event occurs (*e.g.,* push). Moreover, we expect an increase in the adoption trend of SBOMs since, recently (*i.e.,* on March 28, 2023, after our query date), GitHub has introduced a new feature to export SBOMs that allows anyone with read access to a GitHub repository to produce an SPDX SBOM by clicking on a button on the dependency graph page of the repository (GitHub, 2023b). This self-service feature not only enables on-demand SBOM generation but could also be integrated into CI/CD pipelines and build-automation processes. Therefore, we expect that more open-source software projects will adopt SBOMs in the future.

A concern that could arise is the extent to which project owners have decided not to integrate SBOM generation within a CI/CD pipeline, and therefore continuously generate SBOMs because of possible performance overhead. However, considering the SBOM granularity provided by state-of-practice SBOM generators (*i.e.,* package level) and the way such tools work (*i.e.,* by leveraging package managers), once the CI/CD pipeline has resolved the direct and indirect dependencies (necessary to perform the build anyway), then the SBOM generation would take a few seconds even for relatively large projects. The reason is that package managers have already retrieved, during dependency resolution, the required information. In summary, such minimal overhead would be perfectly compatible with CI/CD practices (Duvall et al., 2007). The same applies to SBOM generation using the GitHub feature.

Cleary, if a software producer may need to keep up-to-date SBOMs for software written in programming languages for which there is no

package manager, then such a task may become cumbersome and left to the developers' responsibility.

We also advise the need to promote the integration of SBOM generation tools into the development pipeline. Embedding such tools is important for **software producers** to enhance the transparency and security of their software projects. Recent initiatives including the GitHub's native SBOM generation feature (GitHub, 2023b) represent a promising step towards mainstream adoption, enabling developers to automatically produce and maintain SBOMs as part of their CI/CD pipeline.

Finding #3: Only a few software projects follow recommendations on how SBOMs should be distributed.

According to *NTIA* (NTIA, 2021d), SBOMs should be bundled with every release version and archived by the software producer and be updated at each code change. Furthermore, there is a broad consensus on this matter among the respondents of the Linux Foundation's SBOM readiness survey (The Linux Foundation, 2022b). Nevertheless, it appears that most of the studied projects ignore or are unaware of such recommendations, thus limiting the potential benefits due to the adoption of SBOMs. We advise **software producers** to distribute SBOMs following the NTIA's recommendations (NTIA, 2021d), including delivering updated SBOMs at each code change and release version. Consistent and timely distribution of SBOMs is also essential for maintaining transparency, enabling effective vulnerability management, and supporting **software consumers** in assessing project risks.

Finding #4: The same software project may distribute multiple SBOMs.

Multiple SBOMs are distributed in the same software project because:

(i) Owners/maintainers for the same project generate SBOMs in different formats (*e.g.,* JSON or XML).
(ii) Different SBOMs may be generated to represent the specific hardware or software configurations (*e.g.,* operating systems or processor architectures) required to deploy/run that software.
(iii) SBOMs can be maintained both under version control and in public release versions of a software repository.

SBOM producers should generate SBOMs using widely adopted and well-supported file formats, such as JSON or XML. Since no universally accepted file format exists, using multiple file formats can improve interoperability across the SBOM toolchain. For example, tools such as *sbom-utility* (CycloneDX, 2023e) support only the JSON format. While tools like *cyclonedx-cli* (CycloneDX, 2023d) can convert SBOMs between file formats, whenever possible, SBOMs should be generated natively in the required file formats, ideally using tools that support multi-format outputs (*e.g., cyclonedx-maven-plugin* supports both JSON and XML file formats). Moreover, software projects can distribute SBOMs tailored to each supported hardware or software configuration to accurately represent the components required to deploy/run that software. Generating distinct SBOMs for different deployment targets enhances the precision of the reported components. This practice facilitates more accurate vulnerability management, license compliance, and other SBOM-driven use cases (NTIA, 2021b).

Finding #5: Not all SBOMs are valid with respect to their SBOM standard version, thus potentially compromising the capability of tools consuming SBOMs.

When SBOMs are not valid with respect to their SBOM standard version, tools consuming invalid SBOMs (*e.g.,* vulnerability or license scanners) might fail their ingestion. This would limit the potential benefits due to the adoption of SBOMs. To let **software producers** get the best out of SBOMs (and consequently **software consumers**), we recommend they check the compliance of the SBOMs they produce. To that end, convenient tools like sbom-utility (CycloneDX, 2023e) might come in handy to them. **Software producers** could incorporate SBOM validation tools like sbom-utility into their CI/CD pipelines to ensure that generated SBOMs conform to their declared standard and version (*e.g.,* SPDX 2.3 or CycloneDX 1.5).

Validity errors could originate from inaccuracies in SBOM generation tools or data sources from which information is retrieved to feed SBOMs. For example, a data source could be populated with incorrect information that, if retrieved to feed an SBOM, would make that SBOM not valid with respect to its standard version. In this respect, we foster **researchers** to conduct further investigations.

Finding #6: SBOMs can integrate with software supply chain frameworks.

We found SPDX SBOMs integrating with the *in-toto* attestation framework, which provides a specification for generating verifiable claims about the software supply chain of a software artifact (in-toto, 2023b,a). Frameworks like *in-toto* are also supported by CycloneDX SBOMs (CycloneDX, 2023b). On the one hand, integrating SBOMs with software supply chain frameworks highlights the flexibility to tailor SBOMs according to specific needs. On the other hand, these integrations might lead to deviations in the SBOM schema structure. For example, the integration with *in-toto* added attributes into the SBOMs that fall outside the scope of the standard SBOM specification, potentially leading to errors when these SBOMs are processed by tools expecting strict adherence to the SBOM standard. Ultimately, we encourage **software producers** to integrate SBOM generation in their supply chain management. At the same time, they should carefully manage potential errors that may arise when consuming SBOMs augmented beyond the standard specification.

Finding #7: Most SBOMs do not report all NTIA's minimum data fields; in this respect, the supplier name of components is the least reported data field, and some SBOMs do not report any component information.

NTIA's minimum data fields document baseline information about each component that should be tracked and maintained. This baseline information is needed to uniquely and unambiguously document components and then enable *(i)* the tracking of components across the software supply chain and *(ii)* their mapping to other beneficial sources of data (*e.g.,* vulnerability or license databases) (United States Department of Commerce, 2021). It is easy to grasp that not reporting this baseline information, as we observed, compromises basic use cases like the management of vulnerabilities, software inventory, and licenses, as well as advanced use cases (*e.g.,* software provenance and pedigree). This is clearly of interest to **software producers** and **consumers**. This finding is consistent with previous literature (Torres-Arias et al., 2023; Xia et al., 2023). According to Xia et al. (2023), one of the reasons for not reporting all NTIA's minimum data fields is the unavailability of data to feed SBOMs. However, when this happens **software producers** should explicitly assert it (NTIA, 2021g). Instead, we have witnessed a limited number of non-asserted data fields. We advise **software producers** to treat NTIA's minimum data fields as non-negotiable. In case a data field cannot be provided (*e.g.,* the supplier name is unknown), they should explicitly assert the missing/unavailable information as per NTIA guidelines (United States Department of Commerce, 2021). This would promote transparency for the software system documented by the SBOM. In critical or security-sensitive projects, **software producers** should consider post-processing or enriching generated SBOMs to fill in missing fields, especially in the case of known vulnerabilities.

Regarding the *supplier name* data field, its poor reporting is likely due to the shortage of global identification mechanisms like those used for components (*e.g.,* CPE or purl). Similar to NTIA (2021e), we advise **researchers** to define mechanisms to globally identify component suppliers (similar to those used for components): this should ease the retrieval of information on them from external databases when producing SBOMs.

Finally, we noticed the presence of SBOMs that do not report any component information. This finding suggests a very incorrect and shallow use of SBOMs, which makes these SBOMs less useful for both **software producers** and **consumers**.

Finding #8: Overall, only half of SBOMs report licensing information. The presence of licensing information is more common in SPDX SBOMs than CycloneDX ones.

We can postulate that the problem of poor reporting of licensing information in SBOMs is due to the limitations of SBOM generation tools in retrieving such information. Although we did not perform a comparative study between SPDX and CycloneDX SBOM generation tools, the higher prevalence of licensing information in SPDX SBOMs seems to suggest that SBOM generation tools owned by SPDX are more effective than those owned by CycloneDX in retrieving licensing information to feed SBOMs. This finding is not surprising since the SPDX standard was initially conceived to improve software license compliance, while the CycloneDX standard is developed and maintained by OWASP, which operates in the security context.

Another reason for the poor reporting of licensing information in SBOMs can be the lack of interest in licensing issues by project owners, most of which have been using CycloneDX SBOM generation tools. Our advice to **software producers** is not to underestimate the importance of documenting licensing in their SBOM to facilitate licensing compliance analysis from the software consumer's side, *e.g.,* those who want to integrate a library in their projects. We urge **software producers** to ensure that every component in their SBOMs includes license information and, eventually, to add it manually in case of missing. Neglecting license compliance could expose organizations to legal and financial liabilities, especially in commercial and enterprise contexts where the reuse of open-source components is audited.

Finding #9: A limited fraction of SBOMs report component hashes, those reporting more hash values (using different hashing algorithms) for the same components are even less.

As reported by NTIA (United States Department of Commerce, 2021), robust identifiers are important for mapping the existence of components to external data sources (*e.g.,* vulnerability database). A component hash would provide a foundational element to allow this mapping, besides assisting in instances of component renaming and whitelisting. Component hashes also allow verifying that components have not been altered in an unauthorized fashion (*i.e.,* component integrity verification). Moreover, providing more hashes (using different hashing algorithms) for the same components enhances interoperability—*i.e.,* it increases SBOM compatibility with third-party tools or external data sources working with specific kinds of hashes (Balliu et al., 2023). Summing up, our recommendation to **software producers** is to always provide more than one hash (each generated with a different hashing algorithm) for each component. Increasing SBOM compatibility would be beneficial to **software consumers** as well.

Finding #10: The most reported kind of component identifier in SBOMs is purl, followed by CPE. No component was ever identified via SWID tags.

The higher prevalence of purl in SBOMs, as compared to CPE, is not surprising since CPE identifiers are assigned by a central authority, *National Institute of Standards and Technology* (*NIST*), while purl identifiers are not. This is to say that some components might not yet have CPE identifiers. Nevertheless, we would like to recall that: *(i)* component identifiers like purl, CPE, and SWID tags are foundational elements to retrieve information on components in external data sources; and *(ii)* different data sources can support different kinds of component identifiers—*e.g.,* while NVD exploits CPE for component identification, *Sonatype OSS Index* exploits purl. Similar to our recommendation on component hashes, we suggest reporting software components in SBOMs using different kinds of identifiers. This would be beneficial to **software producers** and **consumers**. We also foster enhancements of SBOM generation and consumption tools to support different kinds of component identifiers.

Finding #11: Most SBOMs (all concerning CycloneDX) do not describe primary components. Also, they do not report primary component nodes within dependency graphs.

According to NTIA (2021a), an SBOM must describe at least one primary component, which represents the subject of the SBOM. The declaration of the primary component is needed to properly document direct and transitive dependencies. Also, an SBOM should contain at least all primary component dependencies (*i.e.,* the direct dependencies), with all transitive dependencies listed or sufficient detail to retrieve transitive dependencies recursively (United States Department of Commerce, 2021). Therefore, reporting information on the primary component is crucial to understand which dependencies are direct or transitive and then adequately support SBOM use cases, such as vulnerability and licensing risk management. We advise **software producers** to list the primary components and their direct dependencies. This would produce benefits for **software consumers** as well.

Finding #12: Although previous work indicated the usefulness of augmenting SBOMs with vulnerability information, such information was never found in the analyzed projects.

It is recommended to store vulnerability information in separate data structures linked to SBOMs (*i.e.,* VEX documents) unless very specific conditions and processes are in place to make SBOMs constantly complete and up-to-date (CycloneDX, 2023a; United States Department of Commerce, 2021). Therefore, we devised four possible reasons for not reporting vulnerabilities in the SBOMs we studied:

1. Information on vulnerabilities, including their exploitability, is stored in separate data structures linked to SBOMs (*i.e.,* VEX documents). However, we preliminarily retrieved the presence of VEX documents conforming to the CycloneDX standard and found no occurrence; accordingly, this reason seems unlikely.
2. Project owners are not interested in publicly disclosing information on vulnerabilities in their software supply chain.
3. Project owners would like to disclose this information, but are worried that vulnerability information can be used as a road map for attackers.
4. SBOM generation tools lack support for retrieving and producing up-to-date information on vulnerabilities in the software supply chain.

The last three points were also raised in past studies (Stalnaker et al., 2024; Zahan et al., 2022; Xia et al., 2023; Fucci et al., 2025a). In particular, regarding the last point, Fucci et al. (2025a) found that information on vulnerabilities might be reported by security tools already in use by open-source software projects, and such information could not be easily integrated into SBOMs (*e.g.,* due to a lack of standardization). Following NTIA recommendations (NTIA, 2021c), we

advise **software producers** to integrate information on vulnerabilities in SBOMs (or by using data structures linked to SBOMs) since attackers can still obtain it in other ways. For example, **software producers** can link SBOMs to vulnerability information using VEX. This would enable the consumption of SBOM and VEX within CI/CD pipelines, which can periodically check for newly disclosed vulnerabilities and alert developers when any risks emerge (Fucci et al., 2025b). We foster **researchers** to improve SBOM generation tools to retrieve information on vulnerabilities and produce better SBOMs.

5.2. Threats to validity

In the following, we discuss the threats that might affect our results. Based on the validity schema by Wohlin et al. (2012), these threats can fall into the following categories: construct, conclusion, and external.[4]

Construct Validity. Threats to construct validity concern the relation between theory and observation (Wohlin et al., 2012). The approach we used to detect the adoption of SBOMs might affect the results. In this respect, we could detect the adoption of SBOMs by using one of the following approaches: *(i)* by identifying the use of SBOM generation tools through the GitHub Dependency Graph or *(ii)* by searching for SBOMs within projects hosted on GitHub. We opted for the first approach because a project could use an SBOM generation tool without versioning any SBOM or attaching SBOMs to any release version (as we observed in 101 out of 186 projects, see Section 4.2). However, the accuracy of the GitHub Dependency Graph might affect the obtained results (Bifolco et al., 2024). To deal with this issue, we ensured that the repositories were actually dependent on the SBOM generation tools by verifying that their dependencies appeared in their dependency graph and package manager files. This resulted in the exclusion of 11 repositories. Another threat is related to the fact that the GitHub Dependency Graph does not support all build-automation tools (*e.g., Gradle* is not supported by the GitHub Dependency Graph but supported by CycloneDX) and programming languages (*i.e.,* those without a package manager like C, which is not supported by the GitHub Dependency Graph but supported by CycloneDX). Last but not least, some projects might still adopt SBOMs, yet there is no visible trace of dependencies towards SBOM generation tools on GitHub (*e.g.,* because developers installed them on their machine), nor any presence of SBOM files in the repository, *e.g.,* because the SBOM is provided when the project releases are redistributed through channels different from GitHub. The metrics used to answer our RQs might pose another threat to construct validity. However, there is no accepted metric to quantitatively assess the adoption of SBOMs. At the same time, NTIA has outlined a set of required and recommended SBOM fields (NTIA, 2021a,f), and therefore, we based our assessment on the coverage of such fields. To some extent, the defined and used metrics represent another scientific contribution of our research.

External Validity. These threats deal with the generalizability of results (Wohlin et al., 2012). The most important threat is that this study is limited to *(i)* open-source projects hosted on GitHub and *(ii)* SBOM generation tools owned by SPDX and CycloneDX. The current study does not observe SBOM practices in closed-source projects. Also, it does not observe SBOM practices and content when the GitHub SBOM generator is used. This is because *(i)* the study is based on data extracted before such a tool was released, and *(ii)* such a tool can be virtually used on any GitHub repository (provided that it has a dependency graph), and therefore searching repositories through dependencies towards SBOM-related tools would not work. We could not examine projects whose package manager was not supported by GitHub's dependency graph, for example, projects developed in C/C++. While the gathered empirical evidence on the adoption, spread, and evolution of SBOMs cannot be considered conclusive, we believe that this evidence can justify further research.

Conclusion Validity. Threats to this kind of validity concern affect the ability to draw the correct conclusions (Wohlin et al., 2012). Our study is exploratory, and therefore, we address our RQs using descriptive statistics and graphical representations. Another threat to conclusion validity relates to the analysis of the SBOM data fields, as it relies on the correct interpretation of the data fields reported in the identified SBOMs. We mitigated this threat by following the mapping of SBOM data fields provided by NTIA (2021a,f). We acknowledge that the adopted SBOM generation tools determined the content of the identified SBOMs (*e.g.,* Yu et al. (2024)), which may have impacted our study results.

6. Final remarks

In this paper, we presented the results of a study investigating the adoption of Software Bill of Materials (SBOMs) by open-source software projects hosted on GitHub. We mined GitHub to identify repositories that adopt SBOM generation tools developed by SPDX (2024b) and CycloneDX (2024b, 2023c). We observed that 186 public repositories adopted SBOMs. For these repositories, we collected repository and owner information and analyzed the evolutionary history. The most important takeaway results from our study can be summarized as follows:

- The adoption of SBOMs by open-source projects is still low, even if there is an increasing trend. This could be due to the growing interest and pressure from major players, such as the United States Government and the European Union.
- The preferred way to generate SBOMs is through build-integration tools, followed by GitHub Actions.
- SBOMs are under version control or available in published release versions in 46% of the software projects analyzed (sometimes the same software project may distribute multiple SBOMs at a time), although it is recommended that SBOMs should be bundled with every release version (and archived by the software supplier) and be updated at each code change.
- 12% of the SBOMs are not valid with respect to their SBOM standard version, only 7% report information on all NTIA's minimum data fields, while none report information on all NTIA's recommended data fields.
- 50% of the SBOMs include licenses (although not asserted in half of them), 38% include hashes, and none provide information on vulnerabilities.

In addition to the future research directions discussed in Section 5, others could meaningfully contribute to advancing our understanding of SBOM usage/generation. For example, it could be interesting to know if the owners of the projects that adopted SBOM generation tools used them in all their projects and why. On this matter, we have to mention that GitHub introduced a new feature to export SPDX SBOMs by leveraging the GitHub Dependency Graph (GitHub, 2023b). This is to say that starting from March 28, 2023 (23 days after concluding our study), new repositories could have automatically adopted SBOM by leveraging this new feature. On the other hand, this also implies that replicating our previous study might lead to inaccurate/erroneous results. Therefore, our research illustrates how a technology grew in popularity to become the standard for open-source projects hosted on GitHub. As additional future work, we plan to investigate the quality of data sources that generation tools exploit to feed SBOMs (*e.g.,* vulnerability databases) and their impact on the produced output (*e.g.,* data fields with invalid information). We would also develop approaches to maintain SBOMs with up-to-date information on components and their

[4] We do not have any threat to internal validity since such threats are defined as influences that can affect the independent variable concerning causality (Wohlin et al., 2012), and we did not investigate causality in our mining study.

data fields (including licenses and vulnerabilities). These approaches could leverage CI/CD methodologies to keep the information reported in SBOMs updated over time. Finally, to gather the perspectives of project maintainers and owners on SBOM adoption, we have planned a new empirical study involving projects that currently do not use SBOMs. In this study, we plan to generate SBOMs for selected projects and submit them via pull requests, inviting project maintainers/owners to accept the contributions and complete a survey on their thoughts on SBOM usage/adoption.

CRediT authorship contribution statement

Sabato Nocera: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation. **Simone Romano:** Writing – review & editing, Writing – original draft, Methodology. **Massimiliano Di Penta:** Writing – review & editing, Writing – original draft, Supervision, Conceptualization. **Rita Francese:** Writing – review & editing, Writing – original draft. **Giuseppe Scanniello:** Writing – review & editing, Writing – original draft, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been financially supported by the European Union NEXTGenerationEU project and by the Italian Ministry of the University and Research MUR, a Research Projects of Significant National Interest (PRIN) 2022 PNRR, project n. D53D23017310001 entitled 'Mining Software Repositories for enhanced Software Bills of Materials (MSR4SBOM)'.

Data availability

We provide a curated dataset Nocera et al. (2024b) about open-source projects hosted on GitHub depending on SBOM generation tools, including information about SBOM adoption. We made this dataset publicly available for replication purposes and to aid research on SBOMs.

References

Balliu, M., Baudry, B., Bobadilla, S., Ekstedt, M., Monperrus, M., Ron, J., Sharma, A., Skoglund, G., Soto-Valero, C., Wittlinger, M., 2023. Challenges of producing Software Bill of Materials for java. IEEE Secur. Priv. 21 (6), 12–23. http://dx.doi.org/10.1109/MSEC.2023.3302956.

Bavota, G., Canfora, G., Di Penta, M., Oliveto, R., Panichella, S., 2015. How the apache community upgrades dependencies: an evolutionary study. Empir. Softw. Eng. 20, 1275–1317. http://dx.doi.org/10.1007/s10664-014-9325-9.

Bifolco, D., Nocera, S., Romano, S., Di Penta, M., Francese, R., Scanniello, G., 2024. On the accuracy of GitHub's dependency graph. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. EASE '24, Association for Computing Machinery, New York, NY, USA, pp. 242–251. http://dx.doi.org/10.1145/3661167.3661175.

Bogart, C., Kästner, C., Herbsleb, J., 2015. When it breaks, it breaks: How ecosystem developers reason about the stability of dependencies. In: Proceedings of IEEE/ACM International Conference on Automated Software Engineering Workshop. IEEE, pp. 86–89. http://dx.doi.org/10.1109/ASEW.2015.21.

Chaora, A., Ensmenger, N., Camp, L.J., 2023. Discourse, challenges, and prospects around the adoption and dissemination of software bills of materials (SBOMs). In: Procedings of International Symposium on Technology and Society. IEEE, pp. 1–4. http://dx.doi.org/10.1109/ISTAS57930.2023.10305922.

CycloneDX, 2023a. Authoritative guide to SBOM - OWASP CycloneDX guide. URL https://cyclonedx.org/guides/sbom/OWASP_CycloneDX-SBOM-Guide-en.pdf/. (Accessed 15 November 2024).

CycloneDX, 2023b. CycloneDX specification overview - recognized predicate type. URL https://cyclonedx.org/specification/overview/#recognized-predicate-type/. (Accessed 15 November 2024).

CycloneDX, 2023c. CycloneDX tool center. URL https://cyclonedx.org/tool-center/. (Accessed 15 November 2024).

CycloneDX, 2023d. CycloneDX/CycloneDX-cli: CycloneDX CLI tool for SBOM analysis, merging, diffs and format conversions. URL https://github.com/CycloneDX/cyclonedx-cli/. (Accessed 15 November 2024).

CycloneDX, 2023e. CycloneDX/sbom-utility: Utility that provides an API platform for validating, querying and managing BOM data. URL https://github.com/CycloneDX/sbom-utility/. (Accessed 15 November 2024).

CycloneDX, 2023f. Specification overview. URL https://cyclonedx.org/specification/overview/. (Accessed 15 November 2024).

CycloneDX, 2024a. History. URL https://cyclonedx.org/about/history/. (Accessed 15 November 2024).

CycloneDX, 2024b. OWASP CycloneDX Software Bill of Materials (SBOM) standard. URL https://cyclonedx.org. (Accessed 15 November 2024).

Duvall, P.M., Matyas, S., Glover, A., 2007. Continuous Integration: Improving Software Quality and Reducing Risk. In: Signature Series (Addison-Wesley), Addison-Wesley Professional, Boston, MA.

Enck, W., Williams, L., 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. IEEE Secur. Priv. 20 (2), 96–100. http://dx.doi.org/10.1109/MSEC.2022.3142338.

European Commission, 2022. EUR-lex - 52022PC0454 - EN - EUR-lex. URL https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A52022PC0454. (Accessed 15 November 2024).

European Union Agency for Cybersecurity, 2020. Guidelines for securing the internet of things - ENISA. URL https://www.enisa.europa.eu/publications/guidelines-for-securing-the-internet-of-things. (Accessed 15 November 2024).

eWeek, 2014. Heartbleed SSL flaw's true cost will take time to tally. URL https://www.eweek.com/security/heartbleed-ssl-flaw-s-true-cost-will-take-time-to-tally/. (Accessed 15 November 2024).

Fucci, D., Di Penta, M., Romano, S., Scanniello, G., 2025a. Augmenting software bills of materials with software vulnerability description: A preliminary study on GitHub. In: International Conference on the Foundations of Software Engineering. ACM.

Fucci, D., Di Penta, M., Romano, S., Scanniello, G., 2025b. Augmenting software bills of materials with software vulnerability description: A preliminary study on GitHub. In: Proceedings of the ACM SIGSOFT International Conference on the Foundations of Software Engineering (FSE 2025) - Ideas, Visions, and Reflections Track. URL https://arxiv.org/abs/2503.13998.

GitHub, 2023a. About the dependency graph - GitHub docs. URL https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/about-the-dependency-graph. (Accessed 15 November 2024).

GitHub, 2023b. Introducing self-service SBOMs - The GitHub Blog. URL https://github.blog/2023-03-28-introducing-self-service-sboms. (Accessed 15 November 2024).

Google Online Security Blog, 2021. Understanding the impact of apache log4j vulnerability. URL https://security.googleblog.com/2021/12/understanding-impact-of-apache-log4j.html. (Accessed 15 November 2024).

Hartmann, H., Trew, T., 2008. Using feature diagrams with context variability to model multiple product lines for software supply chains. In: Proceedings of International Software Product Line Conference. IEEE, pp. 12–21. http://dx.doi.org/10.1109/SPLC.2008.15.

in-toto, 2023a. Attestation/README.md at main · in-toto/attestation. URL https://github.com/in-toto/attestation/blob/main/README.md. (Accessed 15 November 2024).

in-toto, 2023b. Attestation/spec at v0.1.0 · in-toto/attestation. URL https://github.com/in-toto/attestation/tree/v0.1.0/spec/#statement. (Accessed 15 November 2024).

ISO, 2021. ISO/IEC 5962:2021 - Information technology — SPDX® specification V2.2.1. URL https://www.iso.org/standard/81870.html. (Accessed 15 November 2024).

Joinup, 2023. UUID (RFC 4122) - Universally Unique IDentifier (UUID) URN namespace | joinup. URL https://joinup.ec.europa.eu/collection/ict-standards-procurement/solution/uuid-rfc-4122-universally-unique-identifier-uuid-urn-namespace/about/. (Accessed 15 November 2024).

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D., Damian, D., 2014. The promises and perils of mining GitHub. In: Proceedings of Mining Software Repositories. ACM, pp. 92–101. http://dx.doi.org/10.1145/2597073.2597074.

Kloeg, B., Ding, A.Y., Pellegrom, S., Zhauniarovich, Y., 2024. Charting the path to SBOM adoption: A business stakeholder-centric approach. In: Procedings of Asia Conference on Computer and Communications Security. ACM.

Lamb, C., Zacchiroli, S., 2022. Reproducible builds: Increasing the integrity of software supply chains. IEEE Softw. 39 (2), 62–70. http://dx.doi.org/10.1109/MS.2021.3073045.

Levy, E., 2003. Poisoning the software supply chain. IEEE Secur. Priv. 1 (3), 70–73. http://dx.doi.org/10.1109/MSECP.2003.1203227.

Nadgowda, S., 2022. Engram: the one security platform for modern software supply chain risks. In: Proceedings of International Workshop on Container Technologies and Container Clouds. ACM, pp. 7–12. http://dx.doi.org/10.1145/3565384.3565889.

NetworkX, 2023. NetworkX - Networkx documentation. URL https://networkx.org/. (Accessed 15 November 2024).

Nocera, S., Di Penta, M., Francese, R., Romano, S., Scanniello, G., 2024a. If it's not SBOM, then what? How Italian practitioners manage the software supply chain. In: 2024 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 730–740. http://dx.doi.org/10.1109/ICSME58944.2024.00077.

Nocera, S., Romano, S., Di Penta, M., Francese, R., Scanniello, G., 2023. Software Bill of Materials adoption: A mining study from GitHub. In: 2023 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 39–49. http://dx.doi.org/10.1109/ICSME58846.2023.00016.

Nocera, S., Romano, S., Di Penta, M., Francese, R., Scanniello, G., 2024b. On the adoption of Software Bill of Materials in open-source software projects - a replication package. URL https://doi.org/10.6084/m9.figshare.25003811. DOI: 10.6084/m9.figshare.25003811.

Nocera, S., Vegas, S., Scanniello, G., Juristo, N., 2025. Software composition analysis and supply chain security in apache projects: an empirical study. In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories. MSR, IEEE, pp. 103–115. http://dx.doi.org/10.1109/MSR66628.2025.00027.

NTIA, 2021a. Framing software component transparency: Establishing a common Software Bill of Materials (SBOM). URL https://www.ntia.gov/files/ntia/publications/ntia_sbom_framing_2nd_edition_20211021.pdf. (Accessed 15 November 2024).

NTIA, 2021b. Roles and benefits for SBOM across the supply chain | NTIA multi-stakeholder process on software component transparency | use cases and state of practice working group. URL https://www.ntia.gov/files/ntia/publications/ntia_sbom_use_cases_roles_benefits-nov2019.pdf. (Accessed 15 November 2024).

NTIA, 2021c. SBOM myths vs. Facts. URL https://www.ntia.gov/files/ntia/publications/sbom_myths_vs_facts_nov2021.pdf. (Accessed 15 November 2024).

NTIA, 2021d. SBOM options and decision points. URL https://www.ntia.gov/files/ntia/publications/sbom_options_and_decision_points_20210427-1.pdf.

NTIA, 2021e. Software identification | challenges and guidance. URL https://www.ntia.gov/sites/default/files/publications/ntia_sbom_software_identity-2021mar30_0.pdf. (Accessed 15 November 2024).

NTIA, 2021f. Survey of existing SBOM formats and standards. URL https://www.ntia.gov/files/ntia/publications/sbom_formats_survey-version-2021.pdf. (Accessed 15 November 2024).

NTIA, 2021g. The minimum elements for a Software Bill of Materials (SBOM). URL https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf. (Accessed 15 November 2024).

NTIA, 2021h. Vulnerability-exploitability eXchange (VEX) – an overview. URL https://www.ntia.gov/files/ntia/publications/vex_one-page_summary.pdf. (Accessed 15 November 2024).

OWASP, 2021. A06 vulnerable and outdated components - OWASP top 10:2021. URL https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/. (Accessed 15 November 2024).

PyGithub, 2023. PyGithub - PyGithub 2.2.1.dev3+gaf529ab documentation. URL https://pygithub.readthedocs.io/en/latest/introduction.html. (Accessed 15 November 2024).

Spadini, D., Aniche, M., Bacchelli, A., 2018. Pydriller: Python framework for mining software repositories. In: Proceedings of Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, pp. 908–911. http://dx.doi.org/10.1145/3236024.3264598.

SPDX, 2023a. Clause 11: Relationship between SPDX Elements Information - specification v2.2.2. URL https://spdx.github.io/spdx-spec/v2-draft/relationships-between-SPDX-elements/. (Accessed 15 November 2024).

SPDX, 2023b. Clause 5: Composition of an SPDX document - specification v2.3.0. URL https://spdx.github.io/spdx-spec/v2.3/composition-of-an-SPDX-document/. (Accessed 15 November 2024).

SPDX, 2023c. Open source tools - SPDX. URL https://spdx.dev/spdx-tools/. (Accessed 15 November 2024).

SPDX, 2023d. SPDX online tool. URL https://tools.spdx.org/app/convert/. (Accessed 15 November 2024).

SPDX, 2024a. Overview - SPDX. URL https://spdx.dev/about/overview/. (Accessed 15 November 2024).

SPDX, 2024b. SPDX - linux foundation projects site. URL https://spdx.dev. (Accessed 15 November 2024).

Stack Overflow, 2022. Developer survey. URL https://survey.stackoverflow.co/2022/. (Accessed 15 November 2024).

Stalnaker, T., Wintersgill, N., Chaparro, O., Di Penta, M., German, D.M., Poshyvanyk, D., 2024. BOMs away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In: Proceedings of the IEEE/ACM International Conference on Software Engineering. ACM, http://dx.doi.org/10.1145/3597503.3623347.

The European Commission, The organic logo - European Commission. URL https://agriculture.ec.europa.eu/farming/organic-farming/organic-logo_en. (Accessed 10 July 2025).

The Linux Foundation, 2022a. A summary of census II: Open source software application libraries the world depends on - linux foundation. URL https://www.linuxfoundation.org/blog/blog/a-summary-of-census-ii-open-source-software-application-libraries-the-world-depends-on. (Accessed 15 November 2024).

The Linux Foundation, 2022b. The state of Software Bill of Materials (SBOM) and cybersecurity readiness. URL https://www.linuxfoundation.org/research/the-state-of-software-bill-of-materials-sbom-and-cybersecurity-readiness/. (Accessed 15 November 2024).

Torres-Arias, S., Afzali, H., Kuppusamy, T.K., Curtmola, R., Cappos, J., 2019. In-toto: Providing farm-to-table guarantees for bits and bytes. In: Procedings of USENIX Security Symposium. pp. 1393–1410, URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Torres-Arias, S., Geer, D., Meyers, J.S., 2023. A viewpoint on knowing software: Bill of materials quality when you see it. IEEE Secur. Priv. 21 (06), 50–54. http://dx.doi.org/10.1109/MSEC.2023.3315887.

United States Department of Commerce, 2021. The minimum elements for a Software Bill of Materials (SBOM). URL https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf. (Accessed 15 November 2024).

United States Government, 2021. Executive order on improving the nation's cybersecurity | The White House. URL https://bidenwhitehouse.archives.gov/briefing-room/presidential-actions/2025/01/16/executive-order-on-strengthening-and-promoting-innovation-in-the-nations-cybersecurity/. Old: https://www.whitehouse.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/ (Accessed 15 November 2024).

Vu, D.L., Pashchenko, I., Massacci, F., Plate, H., Sabetta, A., 2020. Towards using source code repositories to identify software supply chain attacks. In: Proceedings of ACM SIGSAC Conference on Computer and Communications Security. ACM, pp. 2093–2095. http://dx.doi.org/10.1145/3372297.3420015.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering. Springer.

Xia, B., Bi, T., Xing, Z., Lu, Q., Zhu, L., 2023. An empirical study on Software Bill of Materials: Where we stand and the road ahead. In: Procedings of IEEE/ACM International Conference on Software Engineering3. IEEE, pp. 2630–2642. http://dx.doi.org/10.1109/ICSE48619.2023.00219.

Yu, S., Song, W., Hu, X., Yin, H., 2024. On the correctness of metadata-based SBOM generation: A differential analysis approach. In: 2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 29–36.

Zahan, N., Lin, E., Tamanna, M., Enck, W., Williams, L., 2023. Software bills of materials are required. Are we there yet? IEEE Secur. Priv. 21 (2), 82–88. http://dx.doi.org/10.1109/MSEC.2023.3237100.

Zahan, N., Zimmermann, T., Godefroid, P., Murphy, B., Maddila, C., Williams, L., 2022. What are weak links in the npm supply chain? In: Proceedings of International Conference on Software Engineering: Software Engineering in Practice. ACM, pp. 331–340. http://dx.doi.org/10.1145/3510457.3513044.